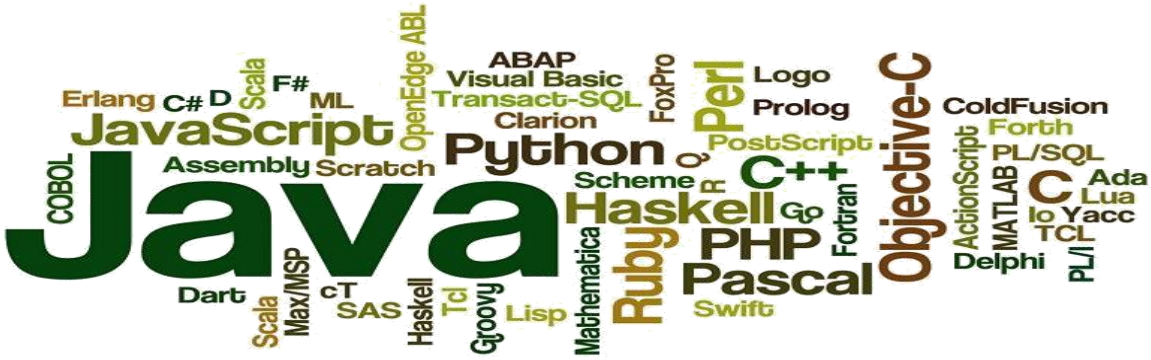


TECNICATURA UNIVERSITARIA EN PROGRAMACION

INGLES I

Docentes: Lic. Nancy N. Rotchen.
Lic. Virginia Vallina

2025

MODULO 1

Texto N° 1:

1. Pre visualice el texto que sigue.

Computer Hope
Free computer help since 1998

What are the differences between hardware and software?

Updated: 06/01/2025 by Computer Hope - https://www.computerhope.com/issues/ch000039.htm

Computer hardware is any physical device used in or with your machine, whereas software is a collection of programming code installed on your computer's hard drive. In other words, hardware is a product you hold in your hand, whereas software cannot be held in your hand. Hardware can be touched, but not software. Hardware is physical and software is virtual. With computer hardware, physical is a device or object that can be touched with your hand. For example, the computer motherboard is a physical device that can be touched when you open your computer case. However, once installed software is not physical and cannot be touched. Virtual describes an object without a physical presence or not what it appears to be. A good example of how virtual describes something is virtual reality. With virtual reality, you can move around in a virtual world and interact with virtual objects and people in that world.

For example, the computer monitor you are using to read this text, and the mouse you are using to navigate this web page are computer hardware. The Internet browser allowing you to view this page and the operating system that the browser is running on are considered software. A video card is hardware, a physical object, and can be touched. A computer game is software and made up of programming code, which is virtual and cannot be touched.

All software uses at least one hardware device to operate. For example, a video game, which is software, uses the computer CPU (Central Processing Unit), RAM (Random-Access Memory), hard drive, and video card to run. Word processing software uses the computer processor, memory, and hard drive to create and save documents.

Hardware is what makes a computer work. A CPU processes information and that information can be stored in RAM or on a hard drive. A sound card provides sound to speakers, and a video card provides an image to a monitor. Each of these are examples of hardware components.

Can a computer run without software?

In most situations, yes, a computer can run without software being installed. However, if an operating system or interpreter is not found, it either generates an error or doesn't output any information. A computer needs an operating system that allows both the user and software to interact with the computer hardware.
Installing programs onto the computer, in addition to an operating system, gives the computer additional capabilities. For example, a word processor is not required, but it lets you create documents and letters.

Can a computer run without hardware?

Most computers require at least a display, hard drive, keyboard, memory, motherboard, processor, power supply, and video card to function properly.
If any of these devices are missing or malfunctioning, an error is encountered, or the computer doesn't start. Adding hardware, such as a disc drive (e.g., CD-ROM (Compact Disc Read-Only Memory) or DVD), modem, mouse, network card, printer, sound card, or speakers are not required, but give the computer additional functionality.

2. Observe el paratexto: el título, los subtítulos, las imágenes, etc. ¿Qué elementos lo conforman?
   _____
   _____
3. Determine la fuente textual: _____
4. ¿Cuál es el tópico? _____
5. ¿Cuáles son los temas que aborda el texto?
   _____
   _____
6. ¿Qué ilustran las imágenes? _____
7. Determine quién escribió este texto, para quién (posibles destinatarios o receptores), para qué (posibles propósitos comunicativos) y cuándo.
   _____
   _____
   _____
8. Ahora, elabore una hipótesis de contenido en base a los elementos paratextuales.
   Ejemplo: *Teniendo en cuenta los elementos paratextuales, el siguiente* ***texto / artículo*** *analizará / abordará / explicará* _____

> ***Una hipótesis de contenido*** *antes de leer un texto es una suposición o* ***predicción*** *sobre lo que tratará el texto, basada en conocimientos previos, la estructura del texto, los elementos del paratexto y otros indicios. Formular hipótesis ayuda a activar el conocimiento previo, establecer un propósito de lectura y mantener el interés.*

9. ¿Cuál es la diferencia esencial entre hardware y software? Elige la opción correcta:
   a. El hardware es caro y el software es barato.
   b. El hardware es un dispositivo físico y tangible, mientras que el software es un conjunto de código virtual e intangible.
   c. El hardware se instala en el disco duro, mientras que el software es el propio disco duro.
   d. El hardware son los programas y el software son los periféricos.

10. ¿Cuál de los siguientes elementos es un ejemplo de software? Elige la opción correcta:
    a) La placa madre (*motherboard*), **b)** El procesador (CPU), **c)** Un navegador de Internet, **d)** Un mouse.

11. Para que un video juego (software) funcione, necesita utilizar varios componentes de hardware. ¿Cuál de los siguientes NO es un componente de hardware mencionado en el texto que un juego utilizaría? Elige la opción correcta:
    a) CPU, **b)** RAM, **c)** Un procesador de texto, **d)** Una tarjeta de video.

12. Según el texto, un procesador de texto es un software indispensable para que la computadora arranque. ¿Verdadero o falso?

13. Explica con tus propias palabras qué significa que el software es “virtual”, utilizando el ejemplo de la realidad virtual que se menciona en el texto.

14. Nombra al menos cinco componentes de hardware que son esenciales para que una computadora pueda funcionar adecuadamente.

15. Describe la relación de dependencia que existe entre el software y el hardware. Utiliza el ejemplo de un procesador de texto para ilustrar tu respuesta.

TEXTO N° 2

Actividades antes de leer el texto:

1. Pre-visualice el texto que sigue. Observe el paratexto y formule una predicción respecto del tema principal del texto o tópico, a partir del título, subtítulos, imagen y tipografía empleada.

   .....

   .....

   .....

2. ¿Cómo está organizado el texto? ¿Qué función cumple cada sección?

   .....

   .....

3. ¿Quién lo escribió, cuándo, para qué (propósito) y para quién/es (posibles destinatarios) fue escrito este texto?

   .....

   .....

4. Complete la siguiente información:
   - Fuente textual: _____
   - Tipo y género de texto: _____
   - La figura 1 ilustra: _____

webopedia TECHNOLOGY CRYPTO DEFINITIONS

Welcome to webopedia: The Internet´s largest educational resource about all things networking and block chain.

Programming Language[1]

LAST UPDATED APRIL 12, 2024 7:59 AM

WRITTEN BY VANGIE BEAL

SHARE

***About the autor**: Vangie Beal is a freelance business and technology writer covering Internet technologies and online business*
since the late '90s.

A programming language is a vocabulary and set of grammatical rules for instructing a computer or computing device to perform specific tasks. The term *programming language* usually refers to high-level languages, such as BASIC, C, C++, COBOL, Java, FORTRAN, Ada, and Pascal.

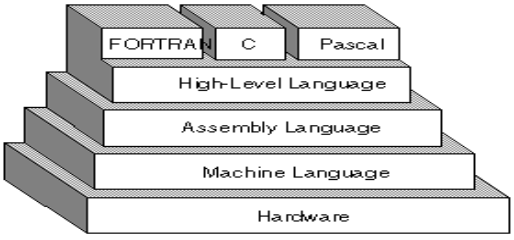


figure 1.

[1] Fuente textual: https://www.webopedia.com/definitions/programming-language/

How does programming language work?

Each programming language has a unique set of keywords (words that it understands) and a special syntax for organizing program instructions.

A program written in a particular programming language has two parts: instructions written in that language and statements written in another language called machine code. Machine code is a binary format that consists of ones and zeros (1 and 0); each digit represents either an instruction or data within the program.

When a programmer types a command into their computer's terminal, it sends those instructions to their computer's processor, which translates them into machine code, so it can execute them. It then takes any information produced by those commands and translates it back into something humans can understand—usually English.

The process is similar for websites; when a user enters text into a search engine like Google, it converts the user's query into machine code before sending it off to its servers. It then processes all of the results from its search algorithm using machine code before translating them back into a human-readable form.

High-Level Programming Languages

High-level programming languages, while simple compared to human languages, are more complex than the languages the computer actually understands, called *machine languages*. Each different type of CPU has its own unique machine language.

Lying between machine languages and high-level languages are languages called assembly languages. Assembly languages are similar to machine languages, but they are much easier to program in because they allow a programmer to substitute names for numbers. Machine languages consist of numbers only.

Lying above high-level languages are languages called *fourth-generation languages* (usually abbreviated *4GL*). 4GLs are far removed from machine languages and represent the class of computer languages closest to human languages. Regardless of what language is used, the program must eventually be converted into machine language so that the computer can understand it. There are two ways to do this:

1) Compile the program.
2) Interpret the program.

What are a programming language's key functions?

The main functions of a programming language are to store and manipulate data, control hardware, provide security, and be an understandable medium for humans. Most programming languages share these functions as their key characteristics but differ in many ways, such as syntax or library sizes.

Other functions vary based on how they're being used. For example, if a language is being used to control hardware, it needs additional programs that tell it how to process data without human interaction.

In addition, encryption may be needed if it's used for communications between two devices. Programming languages use standardized functions to communicate with each other, so compatible devices can communicate.

- **<u>Ahora lea el texto más detalladamente y resuelva las siguientes actividades:</u>**

1) Un lenguaje de programación es _____ para dar instrucciones _____para que _____

2) ¿Qué es un "lenguaje de máquina"? Elige la opción correcta:

   a) Un lenguaje de alto nivel como Java o Python.
   b) Un formato binario (1s y 0s) que la CPU puede ejecutar directamente.
   c) Un lenguaje intermedio que usa nombres en lugar de números.
   d) Un lenguaje similar al lenguaje humano para facilitar la programación.

3) Defina los lenguajes de cuarta generación (4GL), según el texto.

   _____

4) Indica si las siguientes afirmaciones son verdaderas (V) o falsas (F):

   a) Todos los lenguajes de programación tienen la misma sintaxis y tamaño de librerías.

      V / F

   b) Un lenguaje de programación utilizado para comunicaciones entre dispositivos podría necesitar funciones de encriptación. **V / F**

5) Describe el proceso que ocurre cuando un programador introduce un comando en la terminal, desde la escritura del comando hasta que se muestra un resultado legible.

_____
_____
_____
_____

6) Explica con tus propias palabras al menos tres de las funciones clave de un lenguaje de programación descritas en el texto.

_____
_____
_____
_____

7) ¿Por qué cada tipo de CPU tiene su propio y único lenguaje de maquina? ¿Qué implicaciones tiene esto para los programadores que usan lenguajes de alto nivel?

_____
_____
_____

8) Describe la jerarquía de los lenguajes de programación mencionada en el texto, desde el más bajo nivel (cercano al hardware) hasta el más alto nivel (cercano al humano).

_____
_____
_____
_____

9) Lea el texto nuevamente y complete el siguiente cuadro con palabras extraídas del texto:

• 5 palabras transparentes	
• 5 palabras conceptuales conocidas	
• 3 préstamos linguisticos	

*Las palabras **transparentes, o cognados**, son aquellas que tienen una escritura y pronunciación similar o idéntica en dos idiomas diferentes, en este caso, inglés y español. Esto significa que, al encontrarlas en un texto en inglés, es probable que puedas deducir su significado basándote en su forma en español. Agunos ejemplos: program, introduction, to participate, important, etc.*

*Un **préstamo lingüístico** es una palabra o expresión que es adoptada por otra lengua sin traducción directa y conservando su forma y significado original. Algunos ejemplos de palabras en español prestadas del inglés son: software, input, internet y shopping, etc.*

Para lograr una buena comprensión de un texto es necesario lograr una buena interpretación y NO una traducción literal. La misma palabra en inglés puede tener muchos significados; para saber cuál de ellos nos sirve, debemos saber qué función tiene en el texto antes de buscarla en el diccionario. Es decir, debemos conocer la función de una palabra para buscarla en el diccionario con **esa** función, para tener el significado correcto. Debemos averiguar, por ejemplo, si una palabra actúa como sustantivo o verbo para buscar su significado con **esa** función y elegir el adecuado por su relación con el texto. La técnica consiste en dividir el texto en **Frases/Bloques Nominales (FN**) y **Frases/Bloques Verbales (FV).**

¿Qué es una frase / bloque nominal?

- Es una frase o bloque nominal o sustantiva que consiste en un grupo de palabras que engloban una idea, donde el sustantivo común o propio, es el núcleo; el resto de las palabras modifican, o dan información sobre ese núcleo.

La frase nominal está formada por un núcleo (sustantivo o palabra en función sustantiva) y palabras que lo determinan y/o modifican. La frase verbal está formada por un núcleo (verbo conjugado) y las palabras que lo modifican y/o complementan.

Una vez dividido el texto en Frases Nominales y Frases Verbales e identificados sus respectivos núcleos, sabemos de qué estamos hablando y qué acciones se realizan.
Como podemos ver en el gráfico más abajo, la frase o bloque nominal puede estar formada por tres elementos: el núcleo, el pre-modificador (el/los determinadores y los modificadores) y el post-modificador. Una frase sustantiva puede estar formada por una sola palabra, sólo un sustantivo, que es además su núcleo. Si una frase sustantiva está formada por más de una palabra, su núcleo es generalmente la última palabra de la derecha. Para interpretar una frase nominal debemos ubicar el núcleo, buscar su significado y luego interpretar los determinadores y modificadores para que coincidan en género y número con el sustantivo del que se habla en esa frase nominal.

- Observe algunos ejemplos de frases nominales extraídos del texto *"Programming Languages"* de la pág. 5; elige la opción correcta en cada ejemplo:

a. The term **programming language** usually refers to **high-level languages.**

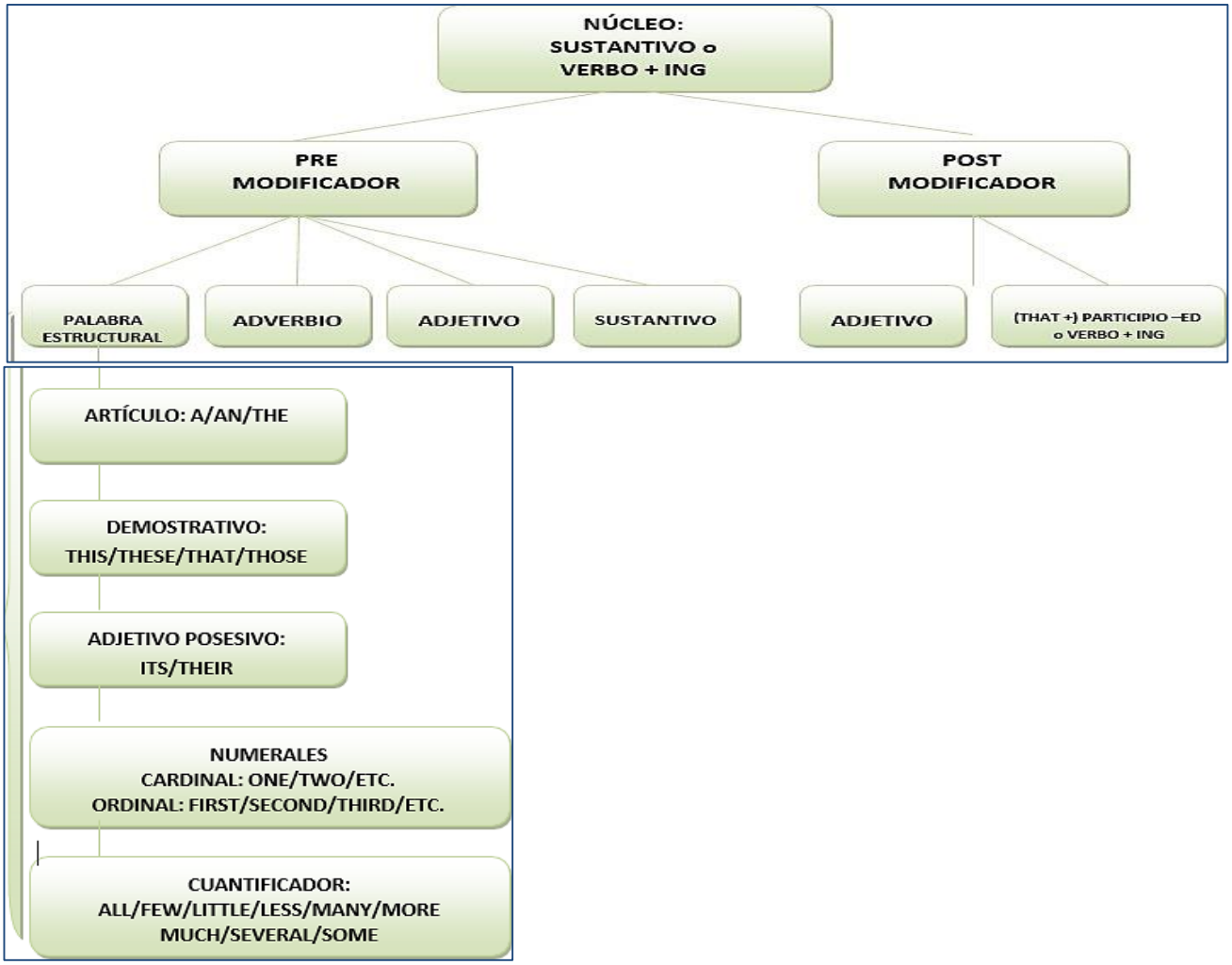
- **programming language:**

El núcleo es: a) programming - b) language
La interpretación más adecuada es: a) la programación lenguaje – b) el lenguaje de programación.

- **High-level languages**

El núcleo es: a) languages – b) high-level
La interpretación más adecuada es: a) lenguajes de alto nivel – b) nivel alto de lenguajes

Estructura de la Frase Nominal:



Ejemplos: (Las palabras en negrita son los núcleos de los bloques nominales).

1. **Python** is a relatively new interpreted programming **language.**
2. **JavaScript** is a high-level, dynamically typed, interpreted **language.**
3. **Java** is **one** of the most popular and widely-used programming **languages** in the **world.**
4. **PHP** is a scripting **language** running on the server **side** to create web **pages** written in **HTML.**
5. **Perl** is a high-level interpreted programming **language** that supports dynamic **programming.**

- Lea los siguientes ejemplos extraídos del texto: "*Programming Languages".* Subraye el núcleo de cada bloque/frase nominal, y luego interprételas:
  a. A special syntax:
  b. Program instructions:
  c. Machine code:
  d. Binary format:
  e. Computer´s processor:
  f. A human-readable form:
  g. A programming language´s key functions:

Ahora, lea las siguientes oraciones extraídas de otros textos. Identifique los bloques verbales en cada oración y analice los bloques nominales. Luego, interprete las oraciones completas.

a) PHP (*Hypertext Preprocessor)* is a widely used open source general-purpose scripting language that is especially suited for web development and can be embedded into HTML.

b) Each high-level language has its own set of predefined words that the programmer must use to write a program.

c) Because the CPU understands only machine language instructions, programs that are written in a high-level language must be translated into machine language.

d) The Java Programming Language is a general-purpose, concurrent, strongly typed, class-based object-oriented language.

TEXTO N° 3: Actividades:

1. **Pre-visualice el texto que sigue**. Observe los elementos del paratexto. Complete la informacion requerida:
   Determine la fuente textual: _____
   Autor/a: _____
   Tipo de texto: _____
   ¿Cuál es el tópico?: _____
   ¿Cuáles son los temas que aborda el texto?: _____
   Explique qué ilustra la imagen: _____

2. **Elabore una hipótesis de contenido** a partir del título, subtítulos, imágenes y tipografía empleada.
   - El texto que voy a leer se trata de _____y va a mencionar _____

3. ¿Quiénes serían los posibles destinatarios de este texto? Justifique su respuesta.
   _____

4. ¿Para qué fue escrito este texto? Indique posibles propósitos comunicativos.
   _____

"Fundamentals of C Programming CS 102 - Introduction to Programming" - By: Dilum Bandara –
Department of Computer Science and Engineering
Faculty of Engineering
University of Moratuwa - Sri Lanka

This section covers the basic concepts of what a programming language is, its purpose, and how it relates to computers and the process of programming.

Chapter 1 – Introduction

1.4 Programming Languages

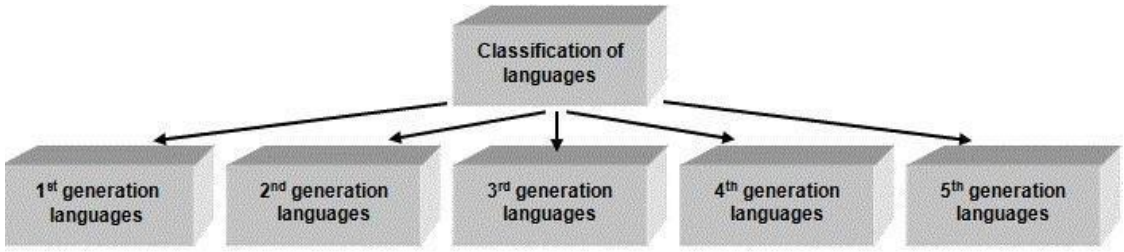
Programming languages were invented to make programming easier. They became popular because they are much easier to handle than machine language. Programming languages are designed to be both high-level and general purpose. A language is high-level if it is independent of the underlying hardware of a computer. It is general purpose if it can be applied to a wide range of situations. There are more than two thousand programming languages and some of these languages are general purpose while others are suitable for specific classes of applications. Languages such as C, C++, Java, C# and Visual Basic can be used to develop a variety of applications. On the other hand, FORTRAN was initially developed for numerical computing, Lisp for artificial intelligence, Simula for simulation and Prolog for natural language processing.

As microprocessors, programming languages can be grouped into several **generations** and currently we are in the **fourth generation**. In the early days, computers were programmed using machine language instructions that the hardware understood directly. Programs written using machine language belongs to the **first generation** of programming languages. These programs were hardly human readable, therefore understanding and modifying them was a difficult task.

Later programs were written in a human readable version of machine code called **Assembly language**. Assembly language belongs to the **second generation** of programming languages. Each Assembly language instruction directly maps into a machine language instruction (there is a 1-to-1 mapping). Assembly language programs were automatically translated into machine language by a program called an assembler. Writing and understanding Assembly language programs were easier than machine language programs. However, even the Assembly language programs tend to be lengthier and tedious to write. Programs developed in Assembly runs only on a specific type of computer.

With the introduction of **third generation** (also referred as 3GL) high-level languages were introduced. These languages allowed programmers to ignore the details of the hardware. The programs written using those languages were portable to more than one type of hardware. A compiler or an interpreter was used to translate the high-level code to machine code. Languages such as FORTRAN, COBOL, C and Pascal belong to the third generation.

All the modern languages such as Visual Basic, VB Script, Java, C# and MatLab belong to the **fourth generation** (4GL). Programs written in these languages were more readable and understandable than the 3GL. They are much closer to natural languages. Source code of the programs written in these languages is much smaller than other generation of languages. However, programs developed in 4GL generally do not utilize resources optimally. They consume large amount of processing power and memory and they are generally slower than the programs developed using languages belonging to other generations. Most of these 4GL support development of Graphical User Interfaces (GUIs) and responding to events such as movement of the mouse, clicking of mouse or pressing a key on the keyboard.
Some people think that **fifth generation languages** are likely to be close to natural languages.
Such languages are one of the major research areas in the field of Artificial Intelligence.



Classifying languages by generation.

Figure 1.

In conclusion:
There are currently five generations of computer programming languages, which show the evolution process of programming language. In each generation, the language syntax has become easier to understand and more human-readable.

First generation languages (abbreviated as 1GL)

Represent the very early, primitive computer languages that consist entirely of 1's and 0's - the actual language that the computer understands (machine language).

Second generation languages (2GL)

Represent a step up from the first generation languages. Allow for the use of symbolic names instead of just numbers. Second generation languages are known as assembly languages. Code written in an assembly language is converted into machine language (1GL).

Third generation languages (3GL)

With the languages introduced by the third generation of computer programming, words and commands (instead of just symbols and numbers) were being used. These languages therefore, had syntax that was much easier to understand. Third generation languages are known as "high level languages" and include C, C++, Java, and JavaScript, among others.

Fourth generation languages (4GL)

The syntax used in 4GL is very close to human language, an improvement from the previous generation of languages. 4GL languages are typically used to access databases and include SQL and ColdFusion, among others.

Fifth generation languages (5GL)

Fifth generation languages are currently being used for neural networks. A numeral network is a form of artificial intelligence that attempts to imitate how the human mind works.

The first two generations are called low level languages. The next three generations are called high level languages.

\- -

5) **Ahora lea el texto más detalladamente y selecciona la opción correcta de acuerdo a la información provista por el texto:**

1. ¿Cuál es el propósito principal de la invención de los lenguajes de programación?

a) Para que las computadoras funcionen más rápido. b) Para hacer la programación más sencilla que usar lenguaje de máquina. c) Para crear interfaces gráficas de usuario (GUI). d) Para procesar lenguaje natural.

2. Un lenguaje se considera de "alto nivel" si...

a) Es muy difícil de aprender. b) Puede aplicarse a una amplia gama de situaciones. c) Es independiente del hardware subyacente de la computadora. d) Se ejecuta de manera más lenta que otros lenguajes.

3. ¿Qué herramienta se utiliza para traducir un lenguaje de ensamblador a lenguaje de máquina?

a) Compilador (Compiler) b) Intérprete (Interpreter) c) Ensamblador (Assembler) d) Depurador (Debugger)

4. Una característica principal de los lenguajes de cuarta generación (4GL) es que:

a) Tienen una sintaxis muy cercana al lenguaje humano. b) Utilizan los recursos del sistema de forma muy óptima. c) Requieren un conocimiento detallado del hardware. d) Cada instrucción se mapea 1 a 1 con el lenguaje de máquina.

6) **Indica si las siguientes afirmaciones son verdaderas (V) o falsas (F), según el texto. En caso de ser falsas, justifica.**

( ) Los programas escritos en lenguaje de máquina son fácilmente legibles y modificables por los humanos.
( ) Los lenguajes de propósito general, como C# o Java, solo pueden usarse para un tipo específico de aplicación.
( ) Un programa escrito en lenguaje de ensamblador es portable y puede ejecutarse en cualquier tipo de computadora sin cambios.
( ) Los compiladores e intérpretes se utilizan para traducir lenguajes de alto nivel a código máquina.
( ) Los lenguajes de cuarta generación (4GL) son generalmente más rápidos y consumen menos memoria que los de tercera generación (3GL).
( ) Los lenguajes de quinta generación (5GL) están asociados con el campo de la inteligencia artificial y las redes neuronales.

7) **Lea el texto y complete los espacios en blanco con la palabra o concepto correcto del texto:**

a) Los lenguajes de primera generación (1GL) consisten enteramente en _____ y _____, que es el lenguaje que la computadora entiende directamente.
b) Un lenguaje es de _____ si se puede aplicar a una amplia variedad de situaciones.
c) Los programas de la tercera generación (3GL) introdujeron lenguajes de _____, que permitían a los programadores ignorar los detalles del hardware.
d) FORTRAN fue un lenguaje desarrollado inicialmente para _____, mientras que Lisp se enfocó en la _____.
e) La mayoría de los lenguajes de cuarta generación (4GL) soportan el desarrollo de _____(GUI) y responden a eventos como el clic del mouse.

8) **Responda las siguientes preguntas en forma clara y concisa, basándote en la informacion y los conceptos del texto:**

a) Explica la relación 1-a-1 (uno a uno) que existe entre el lenguaje de ensamblador (2GL) y el lenguaje de máquina (1GL). ¿Qué implicaciones tenía esto para la programación?

b) Compara y contrasta los lenguajes de tercera generación (3GL) con los de cuarta generación (4GL). Menciona al menos dos diferencias clave discutidas en el texto (por ejemplo, l**egibilidad**, **uso de recursos**).

c) ¿Por qué se considera que los lenguajes de programación han evolucionado para ser "más humanos"? Justifica tu respuesta usando ejemplos de **las diferentes generaciones.**

d) El texto afirma que los programas en 4GL "generalmente no utilizan los recursos de manera óptima". ¿Por qué crees que existe este compromiso entre la facilidad de uso para el programador y la optimización de los recursos del sistema?

TEXTOS INSTRUCTIVOS

La función básica de los textos instructivos es explicar **cómo** desarrollar cierta actividad. Estos tipos de texto suelen seguir una **secuencia ordenada lógica,** facilitando la comprensión y ejecución de tareas. Se utilizan verbos en infinitivo o en modo imperativo para indicar acciones específicas. Los textos instructivos pueden presentarse en diferentes formatos, como guías de usuarios, tutoriales, manuales de referencia, entre otros. Podemos encontrar instrucciones no-verbales, instrucciones que combinan elementos visuales y verbales, y textos instructivos exclusivamente verbales. Ejemplos de frases comunes en textos instructivos técnicos:

- "To install the software, first ***navigate*** to the directory." (Para instalar el software, primero **navegue** al directorio.)
- "Then, ***run*** the setup file." (Luego, **ejecute** el archivo de instalación.)

Actividades:

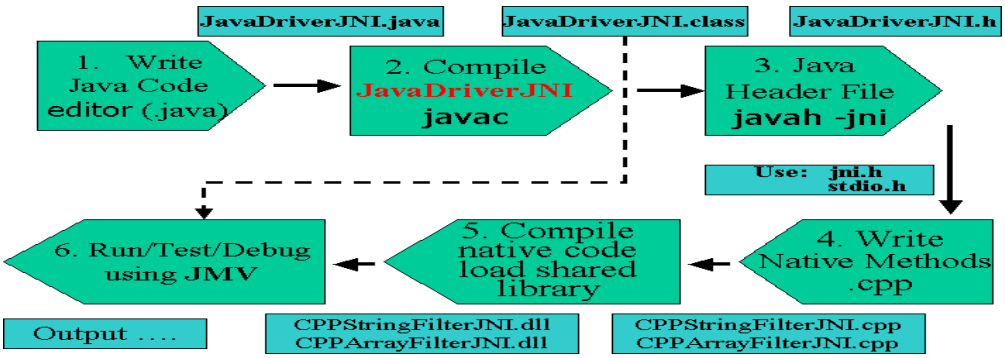
1. Observe el título, los subtítulos y la imagen del texto que va a leer y determine cuál es el tópico del mismo: _____
2. Determine el tipo de texto y los posibles destinatarios: _____
3. Los números indican: _____
4. La función de la imagen es: a) ilustrar - b) informar - c) dar instrucciones.

Writing a Java Program with Native Methods

Writing native methods for Java programs is a multi-step process.

1. Begin by writing the Java program. Create a Java class that declares the native method; this class contains the declaration or signature for the native method. It also includes a main method which calls the native method.
2. Compile the Java class that declares the native method and the main method.
3. Generate a header file for the native method using javah with the native interface flag -jni. Once you've generated the header file you have the formal signature for your native method.
4. Write the implementation of the native method in the programming language of your choice, such as C or C++.
5. Compile the header and implementation files into a shared library file.
6. Run the Java program.

The following figure illustrates these steps for the **JavaDriverJNI.java** program:



Step 1: Write the Java Code

Create a Java class named **JavaDriverJNI** that declares all native methods. This class also includes a main method that creates an object of type **JavaDriverJNI** and calls the native methods, as needed.

Step 2: Compile the Java Code

Use javac **to compile** the Java code that you wrote in **Step 1**.

Step 3: Create the .h File

Use javah to create a JNI-style header file (a .h file) from the **JavaDriverJNI** class. The header file provides a function signature for the implementation of the native methods processString & processArray (these are defines in two different C++ files, see below).

Step 4: Write the Native Method Implementation

Write the implementations for the native methods in native language (such as ANSI C, C++, Fortran) source files (CPPArrayFilterJNI.cpp & CPPArrayFilterJNI.cpp). The implementations will be regular functions **(processArray** & **processString,** respectively) that are integrated with your main Java class, **JavaDriverJNI**.

Step 5: Create a Shared Library

Use the C++ (or other) compiler to compile the header .h files and the source .c files that you created in **Steps 3 and 4** into shared libraries. In Windows 95/NT terminology, a shared library is called a dynamically loadable library (DLL), in Unix these are .so files.

Step 6: Run the Program

And finally, use java **JavaDriverJNI**, the Java interpreter, to run the program.

- ----------

5. **Ahora lea más detalladamente y ordena la siguiente lista de acciones según el flujo de trabajo descrito en el texto:**

   A. Escribe la implementación del método en C++. _____
   B. Ejecute el programa Java. _____
   C. Genere un archivo de cabecera usando javah -jni. _____
   D. Escribe la clase Java que declara el método nativo. _____
   E. Compile los archivos de cabecera e implementación en una biblioteca compartida. _____
   F. Compile la clase Java. _____

6. La frase ***To compile*** en el paso 2 indica:
   - cuándo
   - para qué
   - cómo tomar las precauciones indicadas.

MODULO 2

LAS FUNCIONES RETÓRICAS DEL DISCURSO TÉCNICO-CIENTÍFICO: LA DESCRIPCIÓN

La función retórica descriptiva se clasifica en tres tipos. Cada uno de ellos posee características distintivas y un conjunto de propósitos definido: a) la descripción física, b) la descripción de función, y c) la descripción de proceso.

La descripción física describe las **características físicas de un objeto** y las relaciones espaciales de las partes del objeto entre sí y con todo el conjunto, como así también todo el conjunto con otros objetos relacionados. Las características físicas más frecuentes son la dimensión, el tamaño, el peso, el material, el volumen, el color, y la textura. En general, las descripciones físicas van de lo general a lo particular, con términos como *above* (arriba de), *below* (debajo de), *in the center* (en el centro de), *to the right* (a la derecha de), *near* (cerca de), etc. La manera más sencilla de presentar una descripción física es por medio de una ilustración con indicaciones acerca de las distintas partes o componentes. Tal descripción frecuentemente precede a una descripción de proceso o de función.

La descripción de función involucra el **propósito** que tiene algún dispositivo (o maquinaria) y cómo las partes de esa máquina trabajan u operan de manera independiente, entre sí y con todo el conjunto. Esta clase de descripción está asociada frecuentemente con la relación lógica causalidad / resultado. El tiempo verbal presente se usa en este tipo de descripción.

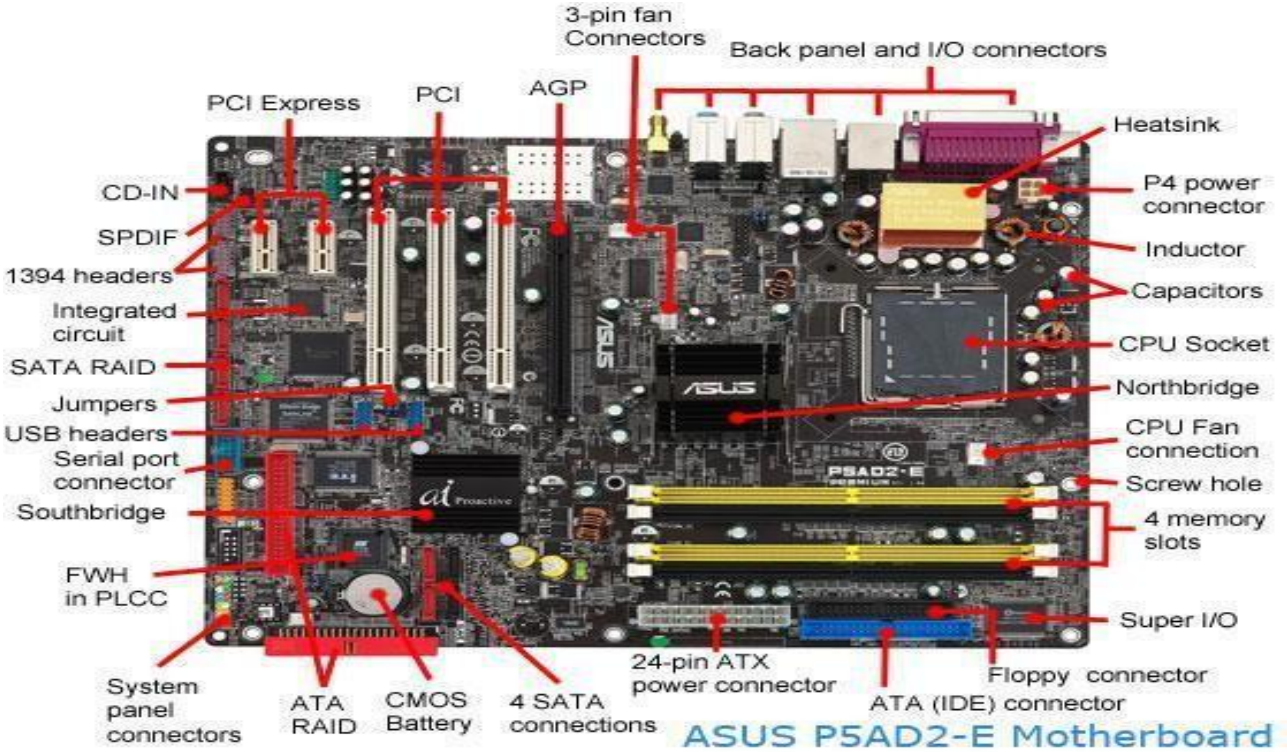
La descripción de proceso se la puede categorizar como un tipo de descripción de función y se refiere a una **serie de pasos o etapas** que se interrelacionan entre sí ya que cada paso (excepto el primero) es dependiente del anterior y así sucesivamente. Además, todos los pasos convergen hacia un objetivo determinado. Este tipo de proceso exige el uso de la forma imperativa del verbo (*imperative form*). También se pueden emplear el verbo modal *should* (debería / es aconsejable) y su forma negativa *shouldn't / should not* (no debería / no es aconsejable) y el verbo modal *can* (puede/n).

Actividades:

A. La descripción física

1) Observe la siguiente descripción de una placa madre (*motherboard):*

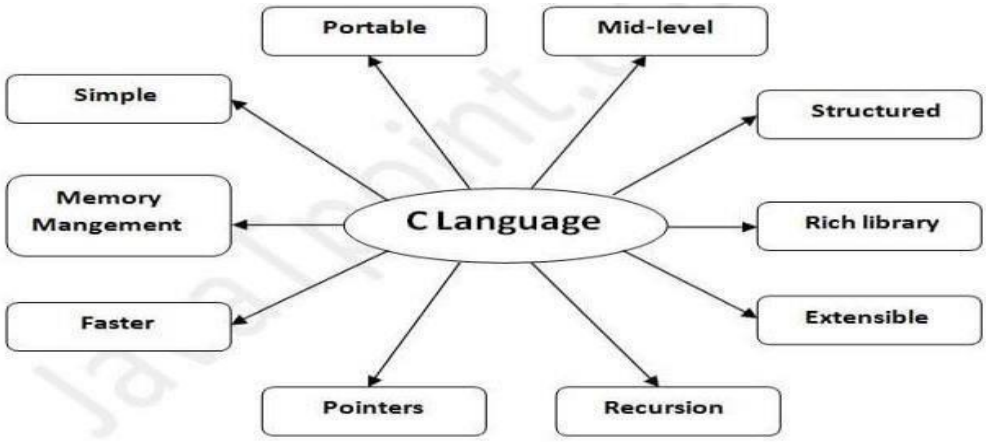
¿Qué muestra esta imagen?
¿Qué se describe?
¿Qué tipo de estructuras predominan en esta descripción física?



2) <u>Observe el siguiente diagrama y el texto que lo acompaña.</u>

1. ¿Qué información brinda este diagrama? ¿Qué relación se establece entre el diagrama y el texto?
2. Subraye en el texto las oraciones o partes de las oraciones que presenten una descripción física.
3. ¿Qué otro tipo de descripción presenta el texto? Subraye en el texto.

Features of C Language



C is the widely used language. It provides many **features**:

1. Simple
2. Machine Independent or Portable
3. Mid-level programming language
4. structured programming language
5. Rich Library
6. Memory Management
7. Fast Speed
8. Pointers
9. Recursion
10. Extensible

1) **Simple**

C is a simple language in the sense that it provides a **structured approach** (to break the problem into parts), **the rich set of library functions**, **data types**, etc.

2) **Machine Independent or Portable**

Unlike assembly language, c programs **can be executed on different machines** with some machine specific changes. Therefore, C is a machine independent language.

3) **Mid-level programming language**

Although, C is **intended to do low-level programming**. It is used to develop system applications such as kernel, driver, etc. It **also supports the features of a high-level language**. That is why it is known as mid-level language.

4) **Structured programming language**

C is a structured programming language in the sense that **we can break the program into parts using functions**. Therefore, it is easy to understand and modify. Functions also provide code reusability.

5) **Rich Library**

C **provides a lot of inbuilt functions** that make the development fast.

6) **Memory Management**

It supports the feature of dynamic memory allocation. In C language, we can free the allocated memory at any time by calling the free () function.

7) **Speed**

The compilation and execution time of C language is fast since there are lesser-inbuilt functions and hence the lesser overhead.

8) **Pointer**

C provides the feature of pointers. We can directly interact with the memory by using the pointers. We **can use pointers for memory, structures, functions, array**, etc.

9) **Recursion**

In C, we **can call the function within the function**. It provides code reusability for every function. Recursion enables us to use the approach of backtracking.

10) **Extensible**

C language is extensible because it **can easily adopt new feature**

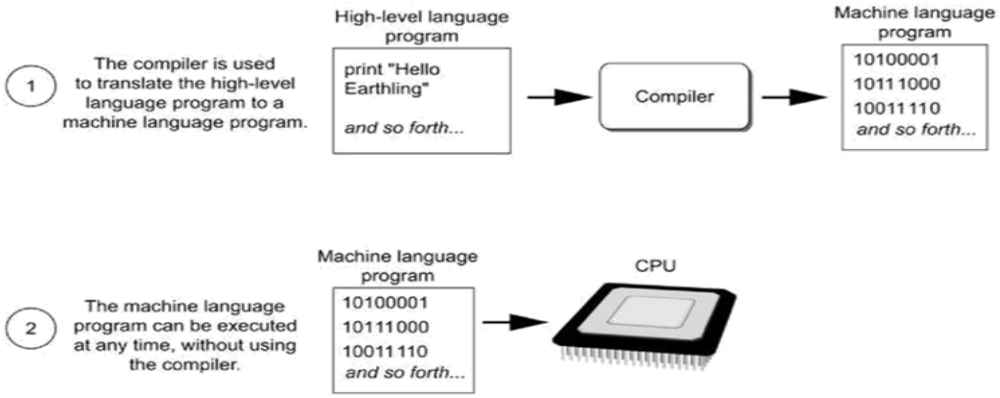
B. La descripción de proceso

La descripción de proceso busca aclarar las fases y/o interrelaciones de una secuencia. Este tipo de descripción también puede ofrecerse a través de diagramas (casi siempre incluyendo flechas para indicar generalmente flujo o movimiento, incluida también la secuencia).

1. Observe los dos diagramas que aparecen a continuación y lea los textos que los acompañan.

Programs are usually stored on a secondary storage device such as a disk drive. When you install a program on your computer, the program is typically copied to your computer's disk drive from a CD-ROM, or perhaps downloaded from a website. Although a program can be stored on a secondary storage device such as a disk drive, it has to be copied into main memory, or RAM, each time the CPU executes it. For example, suppose you have a word processing program on your computer's disk. To execute the program, you use the mouse to double-click the program's icon. This causes the program to be copied from the disk into main memory. Then, the computer's CPU executes the copy of the program that is in main memory. This process is illustrated in Figure 1-16.

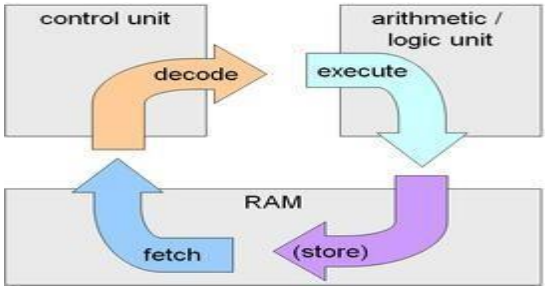
Figure 1-16 A program is copied into main memory and then executed



When a CPU executes the instructions in a program, it is engaged in a process that is known as the *fetch-decode-execute cycle*. This cycle, which consists of three steps, is repeated for each instruction in the program. The steps are:

1. **Fetch** A program is a long sequence of machine language instructions. The first step of the cycle is to fetch, or read, the next instruction from memory into the CPU.
2. **Decode** A machine language instruction is a binary number that represents a command that tells the CPU to perform an operation. In this step, the CPU decodes the instruction that was just fetched from memory, to determine which operation it should perform.
3. **Execute** The last step in the cycle is to execute, or perform, the operation.

Figure 1-17 illustrates these steps.



2. Conteste las siguientes preguntas después de observar con cuidado los diagramas.

a. ¿Los diagramas de proceso suelen ser más sencillos o más complejos que los diagramas de descripción física?

Son más _____

c. ¿Qué se explica en el primer diagrama, con sus propias palabras?

……

……

……

d. ¿Cuáles son los pasos para que la CPU ejecute las instrucciones en un programa?

……

……

……

3. Lea el siguiente texto y complete el cuadro en español:

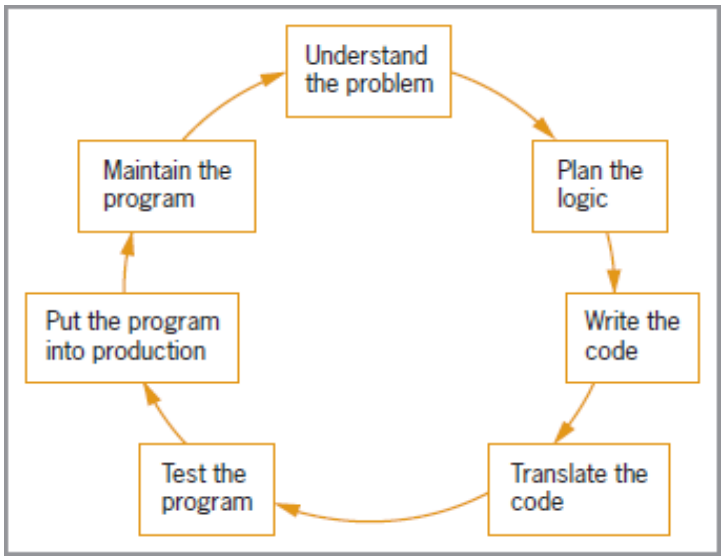
Program development

Program development is the process of creating application **programs**. **Program development** life **cycle** (PDLC), the process containing the five phases of **program development**: analyzing, designing, coding, debugging and testing, and implementing and maintaining application **software**.

The following are six steps in the Program Development Life Cycle:

1. *Analyze the problem*. The computer user must figure out the problem, then decide how to resolve the problem - choose a program.
2. *Design the program*. A flow chart is important to use during this step of the PDLC. This is a visual diagram of the flow containing the program. This step will help you break down the problem.
3. *Code the program*. This is using the language of programming to write the lines of code. The code is called the listing or the source code. The computer user will run an object code for this step.

4. *Debug the program.* The computer user must debug. This is the process of finding the "bugs" on the computer. The bugs are important to find because this is known as errors in a program.
5. *Formalize the solution*. One must run the program to make sure there are no syntax and logic errors. Syntax are grammatical errors and logic errors are incorrect results.
6. *Document and maintain the program*. This step is the final step of gathering everything together. Internal documentation is involved in this step because it explains the reasoning one might of made a change in the program or how to write a program.



Proceso	Tareas a realizar

C. La descripcion de función.

La descripción de función es una descripción del propósito del todo o de las diferentes partes de un elemento, y frecuentemente se encuentra acompañada de una descripción física y/o de proceso.

1. Lea el siguiente texto sobre el funcionamiento de los sistemas de la micro- computadora.

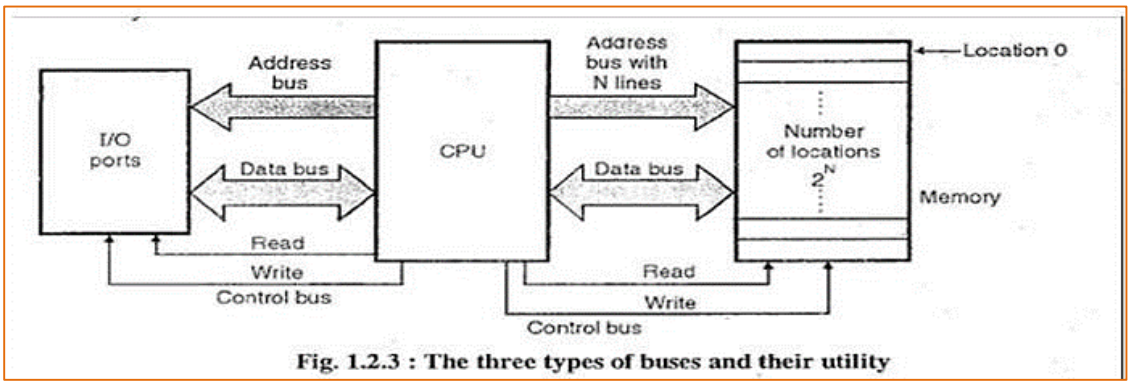


Fig. 1.2.3 : The three types of buses and their utility

The I/O (input/output) unit consists of one or more ICs, which are used to control the data going in and out of the computer.

The ROM (read-only memory) and RAM (random-access memory) units consist of special digital chips, which can store programs and data. The small ROM provides some permanent storage and the RAM is used for temporary storage. Unlike the ROM, the contents of the RAM are constantly changing, but it only operates while the computer is switched on.

The CPU (central processing unit) is a microprocessor. It is the main part of the computer, which controls the rest of the system and performs all the arithmetic and logic operations on the data.

Sets of connectors known as buses are used to carry the internal signals between each unit. The data bus is used to transfer data between all the units. The control bus is used to send control signals from the CPU to the other units. The address bus is used to send signals from the CPU, which indicates the memory and I/O locations to be used.

2. Completa el siguiente cuadro basándote en la información del texto.

Componente	**Función**
Unidad I/O	
ROM	
	Almacena temporariamente
CPU	
Data bus	
Control bus	
	Envía señales desde la CPU que indican la memoria y las ubicaciones I/O a utilizar

3. Subraya con color las partes del texto que indican descripción de función y resalte con otro color distinto, las partes que indican descripción física.

Texto N° 4: Actividad de integración.

a. Pre visualice el texto[2] que sigue. Elabore una hipótesis de contenido a partir del título, los subtítulos y la tipografía empleada.

.....

.....

.....

b. Establezca su fuente textual, sus posibles destinatarios y el / los efectos buscados.

[2] Retrieved and adapted from: https://azure.microsoft.com/en-us/resources/cloud-computing-dictionary/what-is-java-programming-language/

Microsoft | Azure

What is Java? *A Beginner's Guide to Java – Microsoft Azure*

Java is a multiplatform, object-oriented programming language that runs on billions of devices worldwide. It powers applications, smartphone operating systems, enterprise software, and many well-known programs. Despite having been invented over 20 years ago, Java is currently the most popular programming language for app developers.

Java is:
Multiplatform: Java was branded with the slogan "write once, run anywhere" (or WORA), and that still holds true today. Java code written for one platform, like the Windows operating system, can be easily transferred to another platform, like a mobile phone OS, and vice versa without being completely rewritten. Java works on multiple platforms because when a Java program gets compiled, the compiler creates a .class bytecode file that can run on any operating system that has the Java virtual machine (JVM) installed on it. It's typically easy to install JVM on most major operating systems, including iOS, which was not always the case.

Object-oriented: Java was among the first object-oriented programming languages. An object-oriented programming language organizes its code around classes and objects, rather than functions and commands. Most modern programming languages, including C++, C#, Python, and Ruby are object-oriented.

When was Java created?
Java was invented by James Gosling in 1995 while he was working at Sun Microsystems. Though it quickly gained popularity after its release, Java didn't start out as the powerhouse programming language it is today.

Java was designed with a syntax style similar to the C++ programming language so that it would already be familiar to programmers when they started using it. With the slogan "write once, run anywhere" at its core, a programmer could write Java code for one platform that would run on any other platform that had a Java interpreter (i.e., Java virtual machine) installed. With the emergence of the internet and proliferation of new digital devices in the mid-1990s, Java was quickly embraced by developers as a truly multiple platform programming language.

The first public version of Java, Java 1.0, was released in 1996. Within five years, it had 2.5 million developers worldwide. Today, Java powers everything from the Android mobile operating system to enterprise software.

What is the Java programming language used for?
Java is an extremely transferable programming language used across platforms and different types of devices, from smartphones to smart TVs. It's used for creating mobile and web apps, enterprise software, Internet of Things (IoT) devices, gaming, big data, distributed, and cloud-based

applications among other types. Here are some specific, real-world examples of applications that are programmed with Java.

4. ¿Cuál es el principio fundamental detrás del lema "write once, run anywhere" (WORA)?
Elige la respuesta correcta.

a) El código Java solo se puede escribir una vez y no se puede modificar.
b) El código Java escrito en una plataforma puede ejecutarse en otras sin necesidad de reescribirlo por completo.
c) Java es el único lenguaje que funciona en cualquier dispositivo sin excepción.
d) Escribir código una vez es más rápido que escribirlo varias veces.

5. ¿Qué característica principal define a Java como un lenguaje de programación "orientado a objetos"?
Elige la respuesta correcta.

a) Se organiza en torno a funciones y comandos lógicos.
b) Fue el primer lenguaje de programación inventado.
c) Su código se estructura alrededor de clases y objetos.
d) Utiliza una sintaxis simple para principiantes.

6. Indica si las siguientes afirmaciones son verdaderas (V) o falsas (F), según el texto. Justifica solamente las respuestas falsas.

a. Java se puede utilizar para desarrollar aplicaciones web y móviles.
b. Java fue diseñado para ser un lenguaje de programación exclusivo para sistemas operativos Windows.
c. El código Java debe ser reescrito completamente para que funcione en diferentes plataformas.
d. Java es actualmente uno de los lenguajes de programación más populares entre los desarrolladores de aplicaciones.
e. Java permite la creación de aplicaciones solo para dispositivos móviles y no para dispositivos IoT o software empresarial.

7. Explica el concepto de "multiplataforma" en el contexto de Java.
¿Cómo logra Java esta capacidad técnicamente?

_____
_____
_____

8. ¿Por qué fue estratégicamente importante que la sintaxis de Java se pareciera a la de C++ cuando se lanzó?

_____
_____
_____

9. Menciona al menos cuatro tipos de aplicaciones o tecnologías diferentes donde se utiliza el lenguaje Java hoy en día, según lo indicado en el texto.

_____
_____
_____

10. Interprete los siguientes bloques nominales extraídos del texto:

i. the most popular programming language for app developers:

ii. the Windows operating system:

iii. the first object-oriented programming languages:

iv. Most modern programming languages:

v. a truly multiple platform programming language:

MODULO 3

Este módulo es un desdoblamiento del módulo anterior, dado que trata en especial las funciones retóricas asociadas con la descripción: **la definición, la clasificación y la ejemplificación**.
Una definición es una explicación clara y precisa sobre qué es algo, un objeto o un fenómeno.
Su propósito es ayudarnos a entender y diferenciar ese algo del resto, aclarando sus características esenciales.

FUNCIONES RETORICAS DEL DISCURSO TECNICO-CIENTIFICO

1. **LA DEFINICION:**

En los textos informativos es común que en la introducción de un nuevo término o concepto se presente una definición. Las definiciones más comúnmente usadas en los textos técnico-científicos pueden agruparse en ***simples* y *complejas.***

Ejemplos de definiciones ***simples****:*

a) *Computers are electronic machines.*

b) *A variable in any programming language is a named piece of computer memory, containing some information inside.*

La ***definición compleja*** no sólo define un término u objeto de manera completa, sino que además incorpora otras funciones retóricas como la descripción o la clasificación.

Ejemplos de definiciones ***complejas****:*

A computer is an electronic machine capable of executing instructions on data. The distinguishing feature of a computer is its ability to store its own instructions. This ability makes it possible for a computer to perform many operations without the need for a person to type in new instructions each time. Thus, it consists of two major parts, memory and the central processing unit (CPU), which communicate through a set of parallel electrical connections called the bus. The bus also connects to input-output devices such as a screen, a keyboard, and disk drives.

- Las definiciones *simples* se pueden sub-clasificar en: ***formales, semi-formales* y *no formales*** según sean más o menos explícitas y precisas.

- La definición ***formal*** contiene tres componentes (el término a definir + la clase a la cual pertenece el término + una característica o características (o diferencias) que distingue(n) al sujeto de otros miembros de su clase).

Ejemplo: *Micrometers are instruments that are used for measuring small dimensions.*

- La definición ***semi-formal*** sólo contiene el término y las diferencias o características, pero no la clase, ya que es obvia o demasiada amplia. A veces esta definición se parece más a una descripción.

Ejemplo*: C is fast, portable and available in all platforms.*

- La definición ***no formal*** posee el término a definir, pero le faltan las otras características o se define a través de un simple **sinónimo** (“The pigmentation, or coloring, depends on the genes”), de la simple mención de una **clase** a la cual pertenece el sujeto a definir (“Lithium is a metal”), o por medio de **ejemplificación** (“Work in physics means such things as…”).

Los exponentes lingüísticos más recurrentes de la definición en inglés son:

a. **is / are** (es / son)
b. **is / are called** (se llama / se llaman)
c. **is / are defined as** (se (lo / la) define como / se (los / las) definen como)
d. **means / mean** (significa / significan)
e. **is / are known as** (se lo / la conoce como / se los / las conoce como)
f. **refers / refer / is o are referred to as** (refiere(n) a / se lo / la refiere como/ se los/las refiere como)
g. **can be defined as** (se lo/la/los/las puede definir como / puede / pueden definirse como)

- También podemos encontrar definiciones en diccionarios, glosarios o enciclopedias.
- A continuación, se muestra una lista extraída de un glosario de algunos términos de codificación fundamentales y sus definiciones que como futuro programador debes conocer.
- Léelas y resalta con color el exponente lingüístico que marca las distintas definiciones.
  Luego, identifica el tipo de definición y expresa cómo se define en cada caso.

Command: A command is an action to ask a computer to perform. For example, when you click to open a file, you're initiating a command that the computer obeys.

Computer program: A computer program is a collection of codes that tell a computer what to do.

Data: Data refers to any information that a computer can store in its programs.

Debug: Debug refers to examining and removing errors from a program´s source code.

HTML: HTML means HyperText Markup Language.

Machine learning: Machine learning can be defined as a form of artificial intelligence (AI) in which programs learn and improve from experience. An example of machine learning is image recognition, which helps software identify objects and people.

2. **LA CLASIFICACIÓN:**

La clasificación se utiliza frecuentemente como primer paso en una definición. Estas dos funciones retóricas están íntimamente relacionadas ya que para clasificar es necesario hallar algún tipo de definición de los elementos en cuestión. Cuando un autor da una clasificación, puede ser para ayudar a sus lectores a entender mejor un concepto global mediante su desglose o bien para ofrecer información acerca de los miembros de un grupo determinado. También es importante la presencia de la **base de la clasificación,** es decir, el criterio con el cual se realizó la clasificación. En este caso, encontraremos expresiones tales como *"on the basis of", "according to", "as regards", "whether... or not", "including",* etc.

- Una clasificación se considera **completa** cuando provee la siguiente información:

-Los **elementos** o miembros que se clasifican (sujeto de la clasificación),
-La **clase** o **grupo** a la cual pertenecen los elementos,
-La **base** o **criterio** con el cual se realizó la clasificación.

Ejemplos:

a) Depending on the level of abstraction provided by the hardware components, programming languages can be classified into two categories: **High-level language and Low-level language.**
b) Computers are classified according to purpose, data handling and functionality.
According to purpose, computers are either **general purpose or specific purpose**.

- Una clasificación es **parcial** cuando no presenta alguno de los tres tipos de información (miembros, clase, criterio). Puede suceder que no se explicita la base de clasificación.

Ejemplos:

a) There are two categories of low-level language: **machine code and assembly language**.
b) Computers can be classified in many different ways -- by size, by function, or by processing capacity.

- Lee los siguientes textos (A y B) cuya función predominante es la clasificación en cada caso.
En cada texto:

a. Dibuja un círculo alrededor del **sujeto de la clasificación**
b. Indica en **cuántos grupos** se divide la clasificación
c. Indica, donde sea posible, la **base de la clasificación**
d. Indica qué **tipo de clasificación** presentan estos ejemplos.

Ejemplo:

A. Based on the way a system performs the computations and type, a computer can be classified as follows:
Analog Computer
Digital Computer
Hybrid Computer

B. Programming languages are basically classified into two main categories – Low-level language and High-level language.

3. **LA EJEMPLIFICACION:**

Ejemplificar es demostrar con ejemplos y el propósito de esta función retórica es la de:
a) **aclarar** y hacer más accesible un concepto, ó b) **restringir** una categoría.

- Algunos ejemplos de marcadores de ejemplificación: ***for instance, for example, such as, like, including, in other words, as an example, that is,*** **etc.**

- Observa los textos **A y B** a continuación. Subraya el ejemplo propiamente dicho, establezca su función y los conectores o marcadores que se utilizaron en ambos casos.

A. "Alkaline metals (lithium, sodium, magnesium, **for instance**) show a different reaction"

B. "Certain elements **such as** chromium and tungsten may be added to steel in order to improve its property against corrosion and add to its hardness, respectively".

- **<u>Analice los siguientes ejemplos:</u>**

1. The syntax of a programming language is the representation of its programmable entities, **for example**, expressions, declarations and commands.

   Concepto ejemplificado: ……

   Ejemplos: ……..

2. Java is dominating the world of programming because of its primary features **like** security, platform-independent, multi-threading, etc. It is a class-based, server-side, and object- oriented language widely used for back-end development.

   Concepto ejemplificado: ……

   Ejemplos: ……..

3. Due to its versatility, Java can be used for a wide range of tasks **such as** web applications, mobile applications, enterprise applications, embedded systems, scientific computing and much more.

   Concepto ejemplificado: ……

   Ejemplos: ……..

TEXTO N° 5:

- Pre-visualice el texto[3] que sigue. Elabore una hipótesis de contenido a partir del título, subtítulos, las imágenes y la tipografía empleada (palabras en negrita).

_____

_____

- Según sus conocimientos previos, ¿Qué sabe de Python? ¿Por qué es importante aprender este lenguaje?

_____

- Determine la fuente textual: _____
- Determine quién escribió este texto, para quién (posibles destinatarios o receptores), para qué (posibles propósitos comunicativos) y cuándo.

_____

_____

June 2025 - Mónica Martín Rivas -

What is Python?

Python is a high-level, interpreted, general-purpose, easy-to-learn, high-level programming language. Created by Guido van Rossum and first released in 1991, Python has become one of the world's most popular languages and is used in a wide variety of fields, from web and scientific development to artificial intelligence and machine learning.

Advantages of Python

Clear and readable syntax: One of Python's biggest advantages is its simple and readable syntax, which resembles human language. This makes it easy to learn even for those who are new to programming. The clarity of its code encourages cleaner and less error-prone programming.

Versatility: Python is a versatile language that fits a wide range of applications. From web development with frameworks such as Django and Flask, to data analysis with libraries such as Pandas and NumPy, Python has become the preferred choice for many developers due to its ability to tackle a variety of problems efficiently.

[3] *Fuente textual: https://www.domestika.org/en/blog/12504-what-are-the-advantages-of-using-python-as-a-programming-language*

Large number of libraries and frameworks: Python has a vast collection of libraries and frameworks that facilitate the development of different types of projects, from the aforementioned web development tools such as Django and Flask to machine learning libraries such as TensorFlow and PyTorch. Through these resources, developers can work more efficiently and quickly.

Active community: Python has an active and collaborative community of developers worldwide. This community not only contributes to the development and improvement of the language, but also provides valuable support and resources for those learning Python or facing challenges in its development.

Industry adoption: Python has gained widespread popularity in the industry due to its versatility and efficiency. Leading companies such as Google, Facebook, Netflix, and Spotify use Python in their systems and applications, proving its relevance and reliability in large-scale production environments.

Disadvantages of Python

Although Python has many advantages, it also has some limitations that should be taken into account.

Execution speed: Although Python is easy to write and understand, it is not the fastest language in terms of execution speed. This can be a drawback in applications that require high-performance or data-intensive processing.

Memory management: Python uses a garbage collection system to manage memory, which can result in higher resource consumption compared to low-level languages such as C or C++. This can affect performance in applications that require efficient memory management.

Conclusion

In summary, Python offers a unique combination of simplicity, versatility, and power that makes it an attractive choice for a wide range of software development projects. Its clear and readable syntax, extensive collection of libraries and frameworks, active community, and widespread industry adoption are just a few of the reasons why so many developers choose Python as their primary programming language. Whether you're looking to learn how to program or delve into new areas such as artificial intelligence or web development, Python is an excellent choice to begin your journey into the world of programming.

- ----------

1) Lee las siguientes ideas y elige la mejor opción de acuerdo al texto:

a. ¿Quién es el creador de Python?
   a) Dennis Ritchie - b) Bjarne Stroustrup - c) Guido van Rossum - d) James Gosling
b. ¿Cuál de las siguientes es una de las mayores ventajas de la sintaxis de Python?
   a) Es muy compleja y técnica. - b) Se asemeja al lenguaje humano, siendo simple y legible.
   c) Es idéntica a la de C++. - d) Requiere un conocimiento profundo de hardware.

2) Indica si las siguientes afirmaciones son verdaderas (V) o falsas (F), según el texto. Solo justifica las falsas.
   ( ) Python es considerado un lenguaje de bajo nivel.
   ( ) La gestión de memoria en Python es siempre más eficiente en consumo de recursos que en C++.
   ( ) La activa comunidad de Python contribuye al desarrollo del lenguaje y ofrece soporte.
   ( ) Python no es una buena opción para principiantes en programación debido a su sintaxis.
   ( ) Empresas como Netflix y Google utilizan Python en sus sistemas.

3) Explica con tus palabras dos ventajas clave de Python que lo hacen popular.

_____
_____

4) Describe la principal desventaja de Python relacionada con el rendimiento y en qué tipo de aplicaciones podría ser un problema.

_____
_____

5) ¿Por qué la "versatilidad" es considerada una característica importante de Python? Menciona dos campos de aplicación diferentes citados en el texto.

_____
_____

6) Identifica y subraya en el texto la definición de Python. ¿Cómo se define?

_____
_____

7) Completa esta idea:

Los aspectos que hacen que Python sea atractivo para los desarrolladores son:

_____

8) Concéntrese en la sección "***Industry adoption",*** identifique un ejemplo y analice las ideas que une. Indique cuál marcador puede identificar.

_____
_____
_____

9) Identifica una descripción física y otra de función en el texto. Subráyelas y explique qué se describe y cómo se describe en ambos casos.

_____
_____
_____

10) Complete el texto con las frases nominales que correspondan:

a) (Párrafo 1) - Python se ha convertido en uno de _____y se utiliza en _____desde _____hasta _____.

b) (Párrafo 5) - Python tiene _____de _____en _____.

c) (Párrafo 9) - Python utiliza _____para administrar _____, lo que puede resultar en _____ en comparación con _____.

MODULO 4

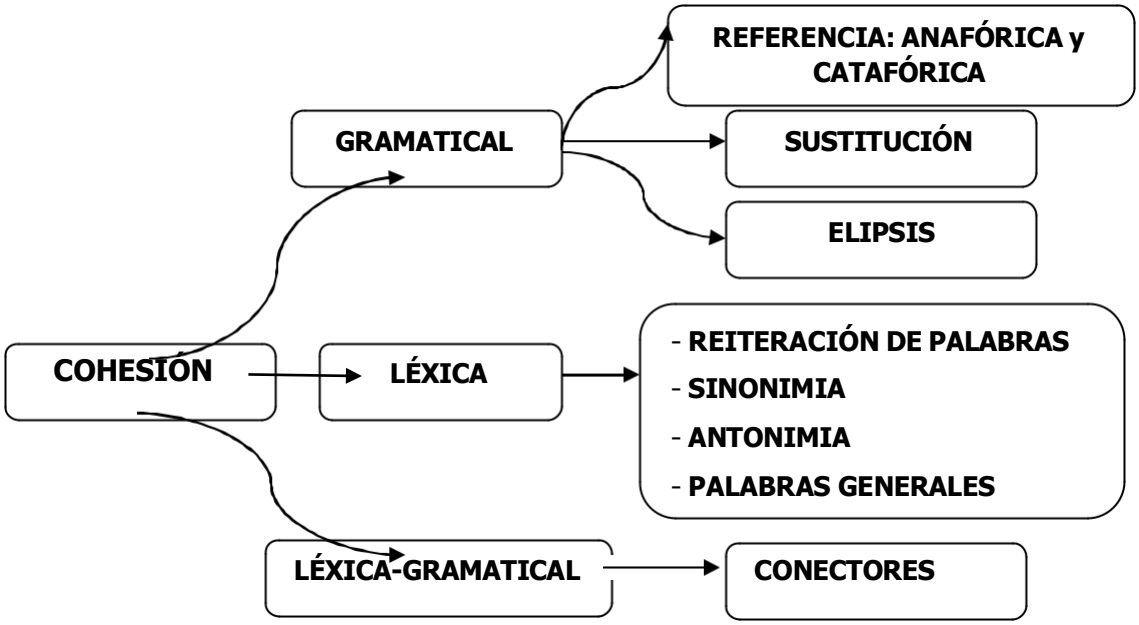
COHESION Y COHERENCIA

Un texto no es una mera suma de oraciones. Para que un texto sea considerado tal, debe contar con dos componentes esenciales, **cohesión y coherencia**, que, si bien se encuentran estrechamente relacionados, son diferentes.

La coherencia puede ser pensada en términos de relación entre los significados y las secuencias de ideas en un texto o en la organización correcta de la información de temas de un texto. Ejemplos típicos serían: problema, solución; pregunta, respuesta.

La cohesión consiste en las conexiones gramaticales y léxicas que conectan una parte de un texto con otro para conformar un todo. Esto incluye el uso de sinónimos, pronombres, tiempos verbales, referencias de tiempo, gramaticales, etc. Por ej.,"it" y "this" refieren a una idea mencionada anteriormente. "First of all", "then"y "after that" ayudan a indicar una secuencia en un texto. "However", "in addition" and "for instance" conectan ideas y argumentos en un texto.

Hay tres clases de cohesión:



a. *COHESION GRAMATICAL:*

REFERENCIA significa la sustitución de un sustantivo por un pronombre. En la referencia existe total identidad entre el sustantivo y el pronombre que lo reemplaza. La referencia puede ser: 1) *anafórica* (cuando hace referencia a la palabra mencionada antes en el texto), 2) *catafórica* (cuando el elemento aparece después de hacer referencia a él) o 3) *exofórica* (hace referencia a factores que no están dentro del texto, sino al contexto situacional).

Ejemplos: 1) There are five basic programming elements or operations: input, output, arithmetic, conditional and looping. Every program uses at least two of **these.**

2) Regardless of what language **they** use, programmers must become adept at specifying exactly what they want the computer to do.

3) Each high-level programming language that **you** know uses one of three translation methods: a compiler, an interpreter, or a hybrid of the two.

SUSTITUCION: es el reemplazo de un término lexical por otro de la misma clase.

Ejemplos:

a) These computers are expensive but I want **one.**
b) I have coffee every morning and he **does** too.

ELIPSIS: es la omisión de un elemento estructural, que puede ser recuperado mediante la relación entre el elemento omitido y algo ya mencionado en el texto.

Ejemplos:

a) Many languages have been developed for attaining diverse works; **some** (languages) are fairly specialized, and **others** (languages) are quite general purpose.
b) This textbook was written with two primary objectives. **The first** is to introduce the C programming language. **The second** is to introduce the basic concepts of software design.

b) *COHESION LEXICAL:*

Está dada por el uso de palabras de referencia generalizada (Ej: *objects, things, stuff, theory, place*), repetición lexical del mismo ítem lexical, la sinonimia (*command, order)*, la antonimia (*tall, short)*

c) COHESION LEXICO-GRAMATICAL:

CONECTORES: son nexos oracionales que establecen relaciones entre las ideas que éstos expresan. Los más comunes en el discurso técnico-científico son los:

- **adversativos o de contraste:** cuando la segunda oración contrasta o niega la primera.
- **copulativos o de adición:** agregan información a la idea anterior SIN contrastar con ella.
- **de causa o razón:** anuncian una causa o razón.
- **de consecuencia:** anuncian un efecto.
- **de secuencia:** marcan un ordenamiento o secuencia de sucesos.
- **temporales o cronológicos:** localizan acciones en el tiempo y en el espacio.
- **de propósito:** marcan el motivo o propósito de las acciones.

CONECTORES

Adversativos (o de Contraste)	Adición	Causa / razón	Consecuencia	Secuencia	Témporo-espaciales	Propósito
However, (Sin embargo)	and (y)	Because (porque)	So (Entonces)	At first, (Al principio,)	When (Cuando)	In order to (para, con el propósito de)
but (pero)	Besides, (Además)	as (como)	Therefore, (Por lo tanto,)	First, (En primer lugar,)	While (Mientras)	So as to
Although/though (Aunque) Even though	Moreover, (Además)	Due to (Debido a)	Thus, (Así,)	Second, (En segundo lugar,)	At the same time, (Al mismo tiempo,)	In order that
Despite (A pesar de)	What's more, (Aún más)	Owing to (Debido a)	As a consequence, (Como consecuencia,)	Then, (Luego,)	After that, (Después de lo anterior)	So that
In spite of (A pesar de que)	Furthermore, (Más aún)	since (ya que)	As a result, (Como resultado de lo anterior)	Next, (Luego)	Before that, (Antes de lo que sigue)	
Nevertheless, (No obstante)	In addition, (Además)	Because of (causa de)	Thereby (por lo tanto)	Finally, (Finalmente,)	As soon as (tan pronto como)	
While /whereas (mientras que)	Also (también) As well as (Como así también)		Consequently (En consecuencia)		Until (hasta)	

Actividades:

1) Las siguientes oraciones han sido seleccionadas de diferentes fuentes porque ejemplifican usos de referencia relativamente sencillos. Léalas cuidadosamente, identifique los referentes mencionados e indique a qué o a quién(s) hacen referencia.

a) Interpreted-language programs are run by software called an interpreter, **which** executes the program's instructions without first compiling **them** to machine.

which .....

them .....

b) Java was originally developed by James Gosling at *Sun Microsystems* (**which** has since been acquired by Oracle Corporation) and released in 1995 as a core component of Sun

Microsystems' Java platform. The language derives much of **its** syntax from C and C++, but it has fewer low-level facilities than either of **them.**

which ……
its ……
them ………..

c) Every programming language comes with **its** own set of rules **that** outline the grammar and spelling within which it operates. **This** is called the syntax of a programming language. **It** specifies the exact combination of symbols and characters **that** you can use to write code in a chosen language that will be understood by the computer.

its .....
that ....
This is called .....
It .....
that you can use .....

d) Despite notational differences, contemporary computer languages provide many of the same programming structures. **These** include basic control structures and data structures. **The former** provide the means to express algorithms, and **the latter** provide ways to organize information.

These .....
The former ……
the latter ……

e) Fundamentally, programs manipulate numbers and text. **These** are the building blocks of all programs. Programming languages let **you** use **them** in different ways by using numbers and text and storing data on disk for later retrieval.

These ……
you ……
them ……

2. <u>Analiza los conectores en los siguientes párrafos e indica de qué tipo son y qué dos ideas unen:</u>

a. There are numerous benefits from learning C; **however**, the most important benefit is that the C programming language is recognized worldwide and used in a multitude of applications, including advanced scientific systems and operating systems. In today's world, a computer programmer needs to be able to communicate with colleagues in different countries. **Therefore**, it's important that even if they don't speak the same verbal language, at least the computer language is understandable to all.

1. **However: es un conector de: ……**

Idea 1: ……

Idea 2: ……

2. **Therefore: es un conector de: ……..**

Causa: ……..

Consecuencia: ……

b. Programming in C is fairly easy **because** it uses basic commands in English. **However**, C is a compiled language so after you type your commands, in order to execute your program, you need to run it through a compiler to transform the human-readable form into machine-readable language.

1. **Because: es un conector de: ……**

Causa: ……..

Consecuencia: ……..

2. **However: es un conector de: ……**

Idea 1: ……..

Idea 2: ……

c. Low-level languages have the advantage that they can be written to take advantage of any peculiarities in the architecture of the central processing unit (CPU). **Thus**, a program written in a low-level language can be extremely efficient, making optimum use of both computer memory and processing time.
However, to write a low-level program takes a substantial amount of time, as well as a clear understanding of the inner workings of the processor itself. **Therefore**, low-level programming is typically used only for very small programs, or for segments of code that are highly critical and must run as efficiently as possible.

1. **Thus: es un conector de: ……..**

Causa: ……..

Consecuencia: ……

2. **However: es un conector de: ……**

Idea 1: ……

Idea 2: ……

3. **Therefore: es un conector de: ……**

Causa: ……

Consecuencia: ……

TEXTO N° 6

- Pre-visualice el texto[4] que sigue. Elabore una hipótesis de contenido a partir del título, subtítulos, las imágenes y la tipografía empleada.

_____

_____

- Según sus conocimientos previos, ¿Cuáles lenguajes de programación son los más adecuados para la IA (Inteligencia Artificial)? ¿Por qué?

_____

- Determine la fuente textual: _____
- Determine quién escribió este texto, para quién (posibles destinatarios o receptores), para qué (posibles propósitos comunicativos) y cuándo.

_____

_____

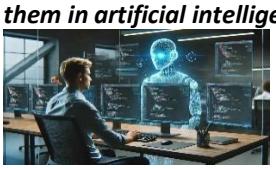
_____

Data > AI and Machine Learning > Python vs. C#: Which Language Is Best for AI?

Python vs. C#: Which Language Is Best for AI?

Written by: Coursera Staff • Updated: 4 may 2025

When choosing a programming language, you want one with useful features for your project. Learn more about the programming languages Python and C#, including how developers use them in artificial intelligence and which language is best for AI projects.



Python and C# are two of the top programming languages used worldwide. They're still in demand as the increasing use of artificial intelligence (AI) creates opportunities for employees with the skills necessary to leverage new technologies and the information they can provide organizations. Businesses can implement artificial intelligence to automate processes, learn about their customers, find ways to improve the products and services they provide, and more.

Computer programming is an important skill for working with artificial intelligence, and some programming languages are better suited for artificial intelligence than others. You'll have to choose the one that meets the specific requirements of the AI project or application you are working on.

What is artificial intelligence?

Artificial intelligence is a branch of computer science designed to research and create the capacity of computers to copy human intelligence. Researchers in this field use algorithms to train computers and computer programs to complete tasks and perform activities that humans handled in the past. Known as machine learning, this process is an important component of several modern technologies **such as** autocorrect, self-driving cars, and chatbots. For example, each time you accept or decline the suggested word spelling from an autocorrect feature, you train the computer behind it to predict the word you want to use and how to spell it.

[4] Fuente textual: https://www.coursera.org/articles/python-vs-c-sharp

The algorithms that power AI can sort through large amounts of data as they seek out patterns and identify trends. With this information, organizations can gain a better understanding of their data, develop accurate predictions, support problem-solving, and make data-driven decisions. Two of the main subfields within artificial intelligence are machine learning and deep learning. Both of them use a type of algorithm called neural networks, which imitate the structure of human brains—complete with layers of interconnected neurons.

Python

Python is a well-known general-purpose programming language and one of the most popular languages used to write code across a variety of disciplines and applications—including artificial intelligence. One reason it's so popular is **because** it uses a syntax similar to English, making it an ideal option for those new to programming. Although Python tends to be easier for people to learn compared to other languages, it still has value for more advanced programmers.

How is Python used in artificial intelligence?

Python is a good choice for use in artificial intelligence **because** it's widely used for data analysis, which is an important component of AI. The language also offers several frameworks and libraries already used in artificial intelligence and machine learning, such as TensorFlow, PyTorch, Keras, and Scikit-learn. For example, developers use TensorFlow to build complex neural networks, automate data, and retrain models. With Python as part of your skill set, you can pursue a career in several AI and AI-adjacent positions, including machine learning engineer, big data analyst, and data scientist.

C#

Part of the C family of languages, which includes C and C++, the object-oriented programming language C# has been in use for more than 20 years. This language is excellent for application development—for the web, desktops, and mobile devices—and Microsoft continues developing it. **As a result**, its capabilities continue to grow as technological advancements occur. C# is a popular and versatile language. If you have prior programming experience with Java or other C languages, it can be helpful to learn C# due to their similarities.

How is C# used in artificial intelligence?

Microsoft developed an open-source machine learning framework called ML.NET to create custom machine learning models. With ML.NET, C# programmers can utilize machine learning to develop applications on mobile and desktop devices, as well as Internet of Things (IoT) applications. Implementing machine learning with app development allows you to add several advanced features to your apps, such as fraud detection, sentiment analysis, object detection, and product recommendations. Additionally, ML.NET offers users access to TensorFlow, the same popular machine learning framework utilized by Python programmers.

Is Python or C# better for artificial intelligence?

In many cases, Python is the best programming language for artificial intelligence, and it is especially useful if you're working on machine learning projects that require large quantities of data. Other features that make Python such a great language for AI are its versatility and the simplicity of the syntax, which make it less challenging to learn. **However**, C# is still a strong option for building mobile, desktop, and web applications, as well as video games, with the ML.NET platform helping to implement machine learning into these systems.

1. Ahora lea el texto más detalladamente y elige la opción más adecuada:

1) **¿Cuál es el propósito principal de la Inteligencia Artificial según el texto?**
a) Reemplazar completamente el trabajo humano.
b) Investigar y crear la capacidad de las computadoras para imitar la inteligencia humana.
c) Desarrollar videojuegos y aplicaciones móviles exclusivamente.
d) Analizar únicamente datos de redes sociales.

2) **¿Qué componente clave de la IA se menciona como un proceso para entrenar computadoras mediante algoritmos?**
a) Desarrollo de aplicaciones. b) Programación orientada a objetos. c) Machine Learning (Aprendizaje Automático). d) Big Data.

2) Identifica si las siguientes afirmaciones son verdaderas (V) o falsas (F). Corrige sólo las falsas.

( ) Python es popular en IA principalmente porque su sintaxis es compleja y similar a la de C++.
( ) C# no tiene ninguna capacidad para implementar Machine Learning en sus aplicaciones.
( ) Los "frameworks" como TensorFlow y PyTorch son herramientas asociadas principalmente a Python para el desarrollo en IA.
( ) ML.NET es un framework desarrollado por Microsoft que permite a los programadores de C# integrar modelos de Machine Learning.

3) Complete la siguiente tabla comparando Python y C# según información proporcionada en el artículo:

Característica	**Python**	**C#**
Facilidad de aprendizaje		
Principal área de aplicación en IA		
Frameworks / librerías de IA mencionados		
Tipos de proyectos donde se destaca		
Empresa / comunidad de respaldo		

4) Menciona dos ejemplos de tecnologías modernas que utilizan Machine Learning, según el texto.
_____
_____

5) ¿Por qué se considera que Python es un lenguaje ideal para quienes se inician en la programación?
_____
_____
_____

6) ¿Para qué tipo de desarrollos se destaca principalmente el lenguaje C#?
_____
_____

7) El texto finaliza con la pregunta: ***"Is Python or C# better for artificial intelligence?".***
Analiza la respuesta que ofrece el autor. ¿Es una respuesta directa?
¿Qué palabras o frases utiliza para posicionar a cada lenguaje?
(Ej.: "En muchos casos, Python es el mejor...", "...C# es todavía una opción fuerte...").
_____
_____
_____

8) ¿Cómo utiliza el autor la popularidad y el respaldo de grandes empresas (como Microsoft) para dar credibilidad a cada lenguaje?
_____
_____
_____

9) Observe las palabras recuadradas en el texto. Determine cuál es el referente y exprese qué información se proporciona.

a) THEY (párrafo 1): ……

b) THEIR (párrafo 4): ……

c) BOTH OF THEM (párrafo 4): ……

10) <u>Complete estas ideas del texto con las frases nominales que correspondan:</u>

a. (Párrafo 5): Python es _____y _____
utilizados para escribir código en _____, incluida _____.

b. (Párrafo 8): Microsoft desarrolló _____
llamado ML.NET para crear _____.

c. (Párrafo 9): C# sigue siendo _____ para crear _____, de _____,
así como también _____, y _____ ayuda a implementar _____
en estos sistemas.

11) ¿Qué **tipo de relación lógica** establecen los siguientes conectores resaltados en **negrita** en el texto? **Completa** las dos ideas que cada uno de ellos conecta.

1. SUCH AS (párrafo 3): es un conector de: ……

   Concepto ejemplificado: ……

   Ejemplos: ……

2. BECAUSE (párrafo 5): es un conector de: ……

   Causa: ……

   Consecuencia: ……

3. BECAUSE (párrafo 6): es un conector de: ……

   Causa: ……

   Consecuencia: ……

4. AS A RESULT, (párrafo 7): es un conector de: ……

   Causa: ……

   Resultado: ……

5. HOWEVER (párrafo 9): es un conector de: ……

   Idea 1: ……

   Idea 2: ……

MODELOS DE PARCIAL ESCRITO

Modelo 1.

1) Pre-visualice el siguiente texto. Elabore una hipótesis de contenido a partir del título y las palabras claves.

2) Lea el siguiente texto sin detenerse en las palabras desconocidas. Explique cómo está organizado el texto y que función cumple cada parte. Ahora resuelva las siguientes actividades en español.

C++ Programming Language – *Last retrieved- June 2022*

C++ is a general-purpose programming language that was developed with the intention to improve C language and include an object-oriented paradigm. Object-Oriented Programming is a programming in which we design and build our application or program based on the object. Objects are instances (variables) of class. It is an imperative and a compiled language. With C++ inheritance, one object obtains all the properties and behaviours of its parent object automatically. It lets you reuse, extend or modify the attributes and behaviours defined in other class.

OOP (Object-Oriented Programming)

C++ is an object-oriented language, unlike C which is a procedural language. It means that

with C++, the main focus is on "objects" and working around these objects. These objects help you execute real-time queries based on inheritance, data abstraction, data encapsulation, data hiding, and polymorphism. Let's discuss what these five main terms of object-oriented programming mean.

Inheritance: Inheritance means inheriting or acquiring the properties and features of the object of one class to another class. It provides reusability of code. The basic idea of inheritance can be implemented by creating more than one class, which we usually refer to as derived classes by joining them with the base class.

Polymorphism: Polymorphism can be referred to as having more than one function with the same name, with different operations. Polymorphism can be static or dynamic. In static polymorphism, memory is allocated at compile time while in dynamic polymorphism, memory is allocated at runtime.

Data Abstraction: The basic notion of data abstraction is to only make the essential information visible and irrelevant information gets hidden from the end-user. Abstraction in programming can hide unnecessary details from the outside world. Using abstraction in our application, the end-user is not bothered even if we modify the internal implementation.

Data encapsulation: Data encapsulation simply implements data abstraction by wrapping up the data and functions into an independent block or single unit called class. Using encapsulation, the C++ software engineer cannot directly access the data as it is only accessible through the object of the class.

Data hiding: Data hiding means shielding data from unauthorized access. It is what secures the data.

Note: Data encapsulation is different from data hiding as encapsulation is about keeping the focus on important data than explaining its composite nature.

3) Describa las principales características y funciones del lenguaje de programación C++.
……
……
……
……

4) Enumere los conceptos principales de la programación orientada a objetos.
……
……
……
……

5) Explique brevemente cada uno de ellos.
……
……
……
……

6) Traslada al castellano los siguientes bloques nominales:

- an imperative and a compiled language:
  ……
  ……
- real-time queries based on inheritance, data abstraction, data encapsulation, data hiding, and polymorphism:
  ……
  ……
  ……
- the properties and features of the object of one class to another class:
  ……
  ……
  ……
- The basic notion of data abstraction:
  ……
  ……
- the C++ software engineer:
  ……

7) ¿A qué hacen referencia las siguientes palabras recuadradas en el texto?

- *WE* (párrafo 1): ……
- WHICH (párrafo 2): ……
- *IT* (párrafo 3): ……
- *ITS* (párrafo 8): ……

8) Localice una definición y subráyela; ¿Qué se define? ¿Cómo se define?

……

……

……

9) Localice una descripción de función y subráyela; ¿Qué se describe? ¿Cómo se describe?

……

……

……

10) Identifique una descripción física. Subráyela en el texto y trasládela al castellano.

……

……

……

Modelo 2:

Lea el siguiente texto y resuelva las actividades en español.

Difference Between Syntax and Semantics

Syntax and Semantics are very significant terms relating to any programming language. The syntax in a programming language involves the set of permitted phrases of a language whereas semantics expresses the associated meaning of those phrases.

Definition of Syntax

The **Syntax** of a programming language is used to signify the structure of programs without considering their meaning. It basically emphasizes the structure, layout of a program with their appearance. It involves a collection of rules which validates the sequence of symbols and instruction used in a program. The pragmatic and computation model figures these syntactic components of a programming language. The tools evolved for the specification of the syntax of the programming languages are regular, context-free and attribute grammars.

However, what is the use of grammar in this aspect? The **Grammars** generally are the rewriting rules whose purpose is to recognize and generate the programs. Grammar does not rely on the computation model instead used in the description of the structure of the language. The grammar

contains a finite set of grammatical categories (such as noun phrase, verb phrase, article, noun, etc.), solitary words (elements of the alphabets) and the well-formed rules to specify the order within which components of the grammatical categories should appear.

Syntax analysis is a task performed by a compiler which examines whether the program has a proper associated derivation tree or not.

Definition of Semantics

Semantics term in a programming language is used to figure out the relationship among the syntax and the model of computation. It emphasizes the interpretation of a program so that the programmer could understand it in an easy way or predict the outcome of program execution. An approach known as **syntax-directed semantics** is used to map syntactical constructs to the computational model with the help of a function.

Key Differences Between Syntax and Semantics

1. Syntax refers to the structure of a program written in a programming language. On the other hand, semantics describes the relationship between the sense of the program and the computational model.
2. Syntactic errors are handled at the compile time. As against, semantic errors are difficult to find and encounter at the runtime.
3. For example, in C++, a variable "s" is declared as "int s;" to initialize it we must use an integer value. Instead of using integer, we have initialized it with "Seven". This declaration and initialization is syntactically correct but semantically incorrect because "Seven" does not represent integer form.

1) ¿A qué se refieren la sintaxis y la semántica de un lenguaje de programación*?*

……

……

……

2) ¿Para qué se utiliza la sintaxis en un lenguaje de programación?

……

……

……

3) ¿Qué es la gramática en un lenguaje de programación?

……

……

……

4) ¿Para qué se utiliza la semántica en un lenguaje de programación?

……
……
……

5) Nombre dos diferencias entre la sintaxis y la semántica.

……
……
……

6) Traslade al castellano los siguientes bloques nominales:
   - the set of permitted phrases of a language:

     _____
     _____
   - a collection of rules which validates the sequence of symbols and instruction used in a program:

     _____
     _____
   - a finite set of grammatical categories:

     _____
     _____
   - the relationship among the syntax and the model of computation:

     _____
     _____

7) ¿A qué hacen referencia las siguientes palabras recuadradas en el texto?
   - IT (párrafo 2): _____
   - WHOSE (párrafo 3): _____
   - WHICH (párrafo 4): _____
   - IT (párrafo 8): _____

8) ¿Qué tipo de relación lógica establecen los siguientes conectores? Sintetice las dos ideas que cada uno de ellos conecta:
   a) Whereas (párrafo 1): …….
   Idea 1: …….
   Idea 2: …….

   *b) Such as (*párrafo 3): …….
   Ejemplo: …….
   Concepto ejemplificado: …….

*c) So that (*párrafo 5): ……
Propósito: ……

De qué? : …….

d) Because (párrafo 8): ……
Causa: ……

Consecuencia: ……..

9) Localice una definición, subráyela, establezca de qué tipo es y trasládela al español.

TRABAJOS PRACTICOS

TP N° 1:

Object-Oriented Programming (OOP)

What is object-oriented programming?

Object-oriented programming (OOP) is a computer programming model that organizes software design around data, or objects, rather than functions and logic. An object can be defined as a data field that has unique attributes and behavior.

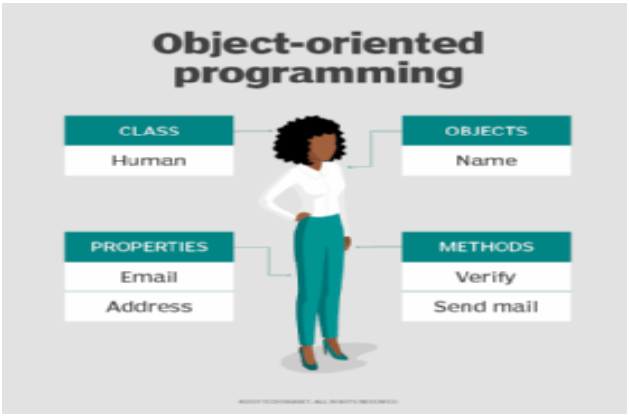
OOP focuses on the objects that developers want to manipulate rather than the logic required to manipulate them. This approach to programming is well-suited for programs that are large, complex and actively updated or maintained. This includes programs for manufacturing and design, as well as mobile applications; for example, OOP can be used for manufacturing system simulation software.

The organization of an object-oriented program also makes the method beneficial to collaborative development, where projects are divided into groups. Additional benefits of OOP include code reusability, scalability and efficiency.

The first step in OOP is to collect all of the objects a programmer wants to manipulate and identify how they relate to each other -- an exercise known as data modeling.

Examples of an object can range from physical entities, such as a human being who is described by properties like name and address, to small computer programs, such as widgets.

Once an object is known, it is labeled with a class of objects that defines the kind of data it contains and any logic sequences that can manipulate it. Each distinct logic sequence is known as a method. Objects can communicate with well-defined interfaces called messages.



This image shows an example of the structure and naming in OOP.

What is the structure of object-oriented programming?
The structure, or building blocks, of object-oriented programming include the following:

- **Classes** are user-defined data types that act as the blueprint for individual objects, attributes and methods.
- **Object**s are instances of a class created with specifically defined data. Objects can correspond to real-world objects or an abstract entity. When class is defined initially, the description is the only object that is defined.
- **Methods** are functions that are defined inside a class that describe the behaviors of an object. Each method contained in class definitions starts with a reference to an instance object. Additionally, the subroutines contained in an object are called instance methods. Programmers use methods for reusability or keeping functionality encapsulated inside one object at a time.
- **Attributes** are defined in the class template and represent the state of an object. Objects will have data stored in the attributes field. Class attributes belong to the class itself.

What are the main principles of OOP?
Object-oriented programming is based on the following principles:

Encapsulation. This principle states that all important information is contained inside an object and only select information is exposed. The implementation and state of each object are privately held inside a defined class. Other objects do not have access to this class or the authority to make changes. They are only able to call a list of public functions or methods. This characteristic of data hiding provides greater program security and avoids unintended data corruption.

Abstraction. Objects only reveal internal mechanisms that are relevant for the use of other objects, hiding any unnecessary implementation code. The derived class can have its functionality extended. This concept can help developers more easily make additional changes or additions over time.

Inheritance. Classes can reuse code from other classes. Relationships and subclasses between objects can be assigned, enabling developers to reuse common logic while still maintaining a unique hierarchy. This property of OOP forces a more thorough data analysis, reduces development time and ensures a higher level of accuracy.

Polymorphism. Objects are designed to share behaviors and they can take on more than one form. The program will determine which meaning or usage is necessary for each execution of that object from a parent class, reducing the need to duplicate code. A child class is then created, which extends the functionality of the parent class. Polymorphism allows different types of objects to pass through the same interface.

Actividades:

1) Pre visualice el texto *"Object Oriented Programming (OOP)".* Elabore una hipótesis de contenido a partir del título, los subtítulos, la imagen y la tipografía empleada.

_____
_____
_____

2) ¿Para qué fue escrito este texto? Indique posibles propósitos comunicativos.

_____
_____

3) Lea e identifique el núcleo de los siguientes bloques nominales extraídos del texto. Luego, trasládelos al castellano.

   a) This approach to programming:

   b) system simulation software:

   c) The organization of an object-oriented program:

   d) Each distinct logic sequence:

   e) methods for reusability or keeping functionality encapsulated inside one object at a time:

   f) unnecessary implementation code:

   g) Relationships and subclasses between objects:

4) ¿A qué hace referencia la sigla OOP? Defina OOP.

_____
_____

5) ¿Cuál es el primer paso en este tipo de programación?

_____
_____

6) ¿Cuál es el siguiente paso y qué se realiza?

_____
_____

7) Explique la estructura básica de OOP.

_____
_____
_____

8) ¿En qué técnicas o principios se basa la OOP? Solamente nómbrelos.

_____

_____

9) ¿Qué diferencias hay entre una clase y un objeto?

_____

_____

10) Identifique una descripción física y otra de función. Subráyelas en el texto. Indique qué se está describiendo en ambos casos.

_____

_____

_____

_____

11) Complete el siguiente cuadro con ejemplos de:

10 palabras transparentes	10 palabras repetidas (relacionadas con el tema principal)

TPN°2

Lea el texto y responda en castellano:

What Is Structured Programming?

Structured programming is a programming paradigm aimed at improving the clarity, quality, and development time of a computer program by making extensive use of subroutines, block structures, for and while loops—in contrast to using simple tests and jumps such as the *go to* statement, which could lead to "spaghetti code" causing difficulty to both follow and maintain.

The principles of structured programming include a technique that takes a top-down design approach with block-oriented structures. A programmer will break a program's source code into logically structured chunks. The main features, dating back to 1966, include sequencing, selection, and iteration.

A Quick Overview

Programming involves the creation of a series of lines, called code, **which** a computer executes. The lines comprise a program. Before structured programs came to the forefront, programmers were frequently guilty of writing "spaghetti code" programs**. This** was due to using the "Go To" statement in programs. With the "Go

To" statement, the program code would be redirected to other areas of the program. Both the Basic and Fortran programming languages had no internal structure or limitations; free form was the rule.

Principles of structured programming help define a "high level" programming language; **these** are human readable language codes. The alternative are low level languages, like Assembler, which are "close" to the hardware level and are full of cryptic symbolic code.

Enter structured programming with its goal to pursue a preferred programming task to make programs produce correct results, with as little redundancy as possible. A structured programming language program is written as a series of explicit statements. It generally does not contain the "Go To" command.

Writing structured programming can be less time consuming. If there is no need to write complex classes or functions, time will not be spent in developing them. Writing classes can take up more time and energy than just writing a piece of code, which executes a specific set of commands. However, this depends on the overall purpose of the program. With structured programming, the separate pieces fall into place, so long as the pieces are well defined.

Principles of Structured Programming

The base composition of any type of structured programming includes three fundamental elements. The first is sequencing, which has to do with the logical sequence provided by the statements in the program. As **they** are executed, each step in the sequence must logically progress to the next without producing any undesirable effects.

The second element is selection. **This step** allows the selection of any number of statements to execute in the program. These statements will contain certain keywords that can identify the sequence as a logically ordered executable. These terms are “if," “then," “endif," or “switch."

A third element is repetition. As a program proceeds, a select statement continues to be active until the program gets to the point where some other action needs to take place. The keywords include "repeat," “for," or “do…until." The repetition factor dictates instructions to the program about how long to continue the operation before requesting further instructions.

Depending on the purpose and function of the program, the exact nature of structured programming will vary. For example, most forms of structured programming will have a single entry point but may have more than one exit point. In another form, called modular programming, the creation of modules within the overall structure of the program will interact with one another, depending on the type of code that is executed.

a. ¿Cuál es el tema principal de este texto?

b. ¿Qué temas aborda el texto?

c. Explique la técnica que se utiliza en el lenguaje estructurado.

d. Nombre por lo menos cinco características que posee el lenguaje estructurado.

e. ¿Qué es un código?

f. ¿Qué elementos componen los lenguajes estructurados? Explique cada uno.

g. Localice una definición, subráyela y trasládela al castellano.

h. ¿En qué casos puede variar la programación estructurada?

i. <u>Traslade al castellano los siguientes bloques nominales:</u>

1. a program's source code into logically structured chunks:
2. human readable language codes:
3. A structured programming language program:
4. the creation of modules within the overall structure of the program:
5. a specific set of commands:
6. The base composition of any type of structured programming:

j. <u>¿A qué hacen referencia los siguientes términos?</u>

1. **Which:**
2. **This:**
3. **These:**
4. **They:**

TP N° 3:

Operating System

The operating system (OS) is the most important program that runs on a computer. Every general-purpose computer must have an operating system to run other programs and applications. Computer operating systems perform basic tasks, such as recognizing input from the keyboard, sending output to the display screen, keeping track of files and directories on the storage drives, and controlling peripheral devices, such as printers.

For large systems, the operating system has even greater responsibilities and powers. It makes sure that different programs and users running at the same time do not interfere with each other. The operating system is also responsible for security, ensuring that unauthorized users do not access the system.

A Software Platform for Applications

Operating systems provide a software platform on top of **which** other programs, called application programs, can run. The application programs must be written to run on top of a particular operating system. Your choice of operating system, **therefore**, determines to a great extent the applications you can run. For PCs, the most popular operating systems are DOS, OS/2, and Windows, but **others** are available, such as Linux.

Classification of Operating systems

- **Multi-user**: Allows two or more users to run programs at the same time. Some operating systems permit hundreds or even thousands of concurrent users.
- **Multiprocessing**: Supports running a program on more than one CPU.
- **Multitasking**: Allows more than one program to run concurrently.
- **Multithreading**: Allows different parts of a single program to run concurrently.
- **Real time**: Responds to input instantly. General-purpose operating systems, such as DOS and UNIX, are not real-time.

User Interaction with the OS

As a user, **you** normally interact with the operating system through a set of commands. For example, the DOS operating system contains commands such as COPY and RENAME for copying files and changing the names of files, respectively. The commands are accepted and executed by a part of the operating system called the command processor or command line interpreter. Graphical user interfaces allow you to enter commands by pointing and clicking at objects that appear on the screen**.**

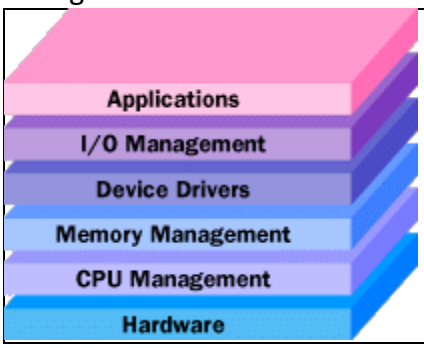
Operating System Functions

At the simplest level, an operating system does two things:

1. It manages the hardware and software resources of the system. In computers, tablets and smartphones these resources include the processors, memory, disk space and more.
2. It provides a stable, consistent way for applications to deal with the hardware without having to know all the details of the hardware.

The first task, managing the hardware and software resources, is very important, as various programs and input methods compete for the attention of the central processing unit (CPU) and demand memory, storage and input/output (I/O) bandwidth for their own purposes. In this capacity, the operating system plays the role of making sure that each application gets its necessary resources while playing nicely with all the other applications, as well as using the limited capacity of the system to the greatest good of all the users and applications.

The second task, providing a consistent user interface, is especially important if there is more than one of a particular type of computer using the operating system, or if the hardware making up the computer is ever open to change. A consistent application programming interface (API) allows a software developer to write an application on one computer and have a high level of confidence that it will run on another computer of the same type, even if the amount of memory or the quantity of storage is different on the two machines.

	*The operating system controls every task your computer carries out and manages system resources to optimize performance.*

Actividades:

a) Aplicando la técnica de lectura rápida: *skimming*, determine el tópico / tema principal del texto.

b) ¿Cómo está organizado el texto y qué función cumple cada parte?

c) Aplicando la técnica de lectura: *scanning*, determine a qué hacen referencia los siguientes índices tipográficos:

1) OS: ……
2) CPU: ……
3) I / O: ……
4) API: ……

d) ¿Qué es un sistema operativo y qué función cumple en una computadora?

e) Defina los programas de aplicación.

f) ¿Cuáles son los sistemas operativos más populares?

g) ¿Cómo se clasifican los sistemas operativos? Explique cada uno brevemente.

h) Nombre y explique las dos tareas importantes que realiza el sistema operativo.

i) ¿A qué hacen referencia los siguientes términos en negrita en el texto?:

1. Which (párrafo 3): ……
2. Others (párrafo 3): ……
3. You (párrafo 5): ……

j) Analice los siguientes conectores lógicos recuadrados en negrita en el texto: qué tipo de conectores son y sintetice las dos ideas que conectan:

a) **FOR EXAMPLE**. Conector de: ……
Ejemplo: _____

Concepto ejemplificado: _____

b) **ALSO**. Conector de: ……

Idea 1: _____

Idea 2: _____

c) **THEREFORE.** Conector de: ……..

Consecuencia: _____

Causa: _____

d) **BUT**. Conector de: ……

Idea 1: _____

Idea 2: _____

PARTE COMUNICATIVA

MEETING PEOPLE:

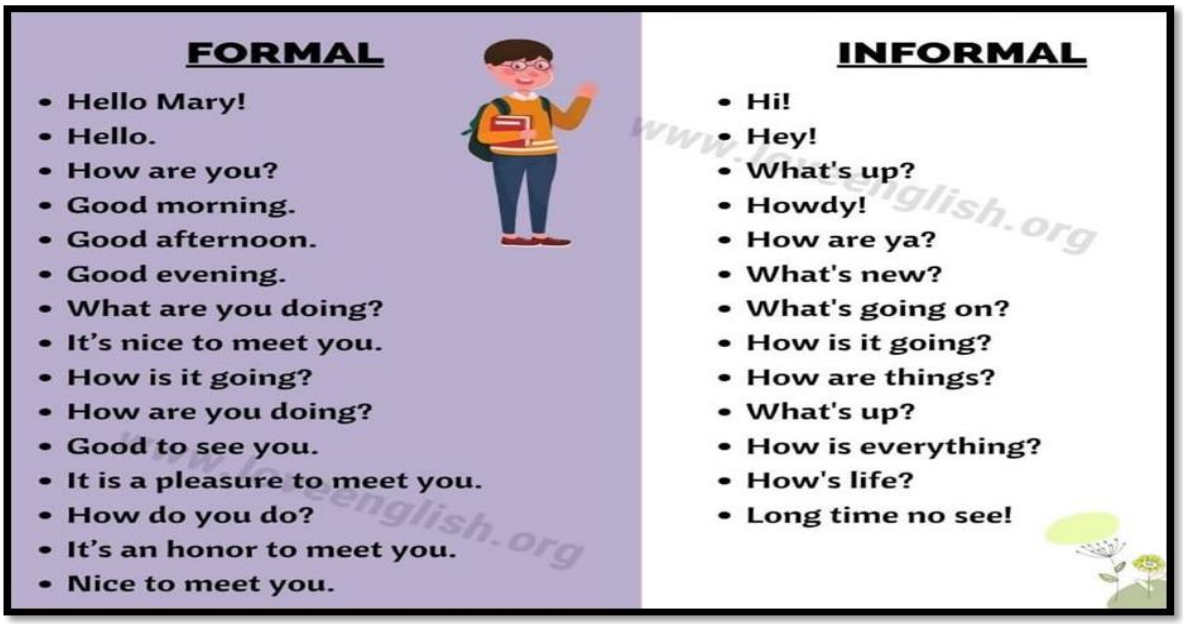
a) Speaking: how do you greet people in your country?
What do you say when you greet people in English?

Look at these three pictures. what is happening in the photos? How do they greet?



Read these ideas and choose the correct option:

- In the first photo, the two men are .....; it´s a **formal / informal** greeting.
- In the second photo, a woman is ..... with a man; the context is meeting **after a long time / for the first time**.
- In the third photo, the two men are.....; they **don´t know / know** each other.



Examples:

Greeting a friend:	Greeting an acquaintance:	Greeting a stranger:
1. Hello! How are you doing? 2. It has been a long time since we last met! 3. Hello! How are you doing now? 4. How's life?	1. Hello! How are you? 2. Hello! How have you been? 3. Good morning/afternoon/evening! All's well?	1. Hello! 2. Good morning/afternoon/evening! 3. How do you do? 4. My name is Aarti. May I know your name?

b) Reading: complete these three dialogues with the words in the box:

all is meet name's Nice this too you Welcome What's

1 Natasha: Hi, my (1) _____ Natasha.
Khalid: Pleased to (2) _____ you. I'm Khalid Ali.
Natasha: Pleased to meet you, (3) _____ .

2 Philip: Good morning. (4) _____ your name?
Ahmed: I'm Ahmed. And (5) _____ are?
Philip: My name's Philip. (6) _____ to meet you.

3 Tim: Hi everybody, (7) _____ is Ingrid.
All: Hi!
Tim: Ingrid, this (8) _____ Ahmed, Linda, Mohammed and Mansoor.
Ingrid: Nice meeting you (9) _____ .
Linda: Likewise.
Tim: (10) _____ to the team and good luck.

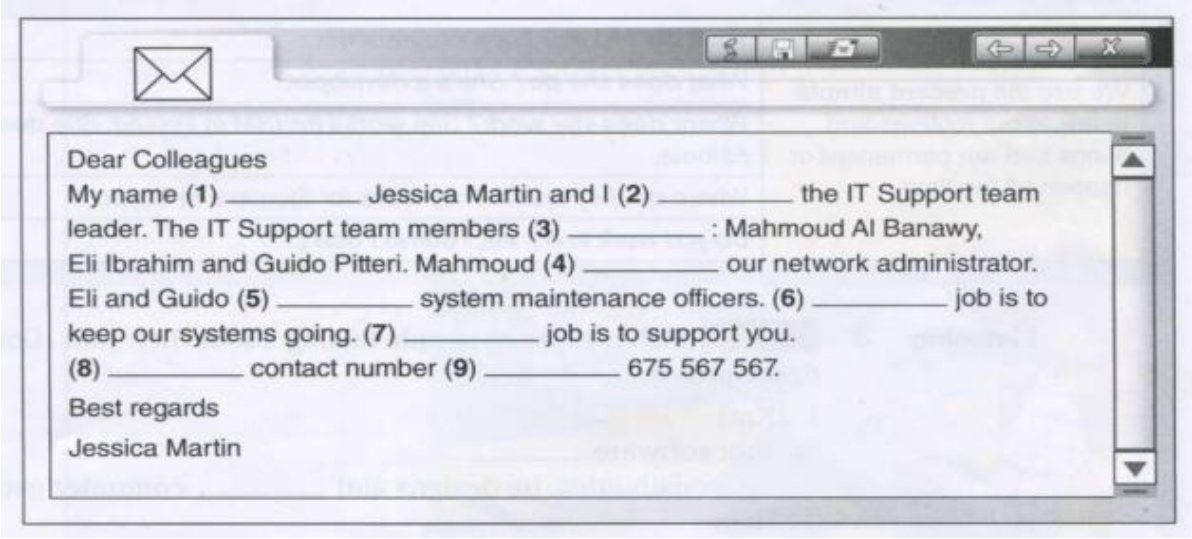
c) Speaking. Practise introductions in small groups.

Situation 1: You have met a friend after a long time. The last time you saw him was in school. **How would you greet each other?**
Situation 2: You meet your local grocer while you are taking your morning walk. **You ask him if his shop will be open today.**
Situation 3: You see a person come down the stairs of your building. You have not seen him before, but you guess that the person may be your new neighbour. **You want to greet him and introduce yourself.**

d) Listening. Listen to this dialogue and choose the correct answers:

Kathryn: Karim, what do you do?
Karim: I'm a (1) *website developer/network administrator*. Who do you work for?
Kathryn: I work for CISCO. I'm a (2) *system analyst/website analyst* there. Where are you from, Karim?
Karim: I'm from Kuwait. I work for Microsoft there. And where are you from, Kathryn?
Kathryn: I'm from the (3) *UK/US* but now I live in Qatar. Do you know where Glenda's from?
Karim: She's from the US.
Kathryn: And what's her job?
Karim: She works for (4) *IBM/Dell*. Her job is to set up new systems.

e) Reading. Complete this email with: ***am, is, are, their, our, my.***



Dear Colleagues
My name (1) _____ Jessica Martin and I (2) _____ the IT Support team leader. The IT Support team members (3) _____ : Mahmoud Al Banawy, Eli Ibrahim and Guido Pitteri. Mahmoud (4) _____ our network administrator. Eli and Guido (5) _____ system maintenance officers. (6) _____ job is to keep our systems going. (7) _____ job is to support you.
(8) _____ contact number (9) _____ 675 567 567.
Best regards
Jessica Martin

f) Writing. Write a reply to the previous email. Introduce yourself and three people in your group.

Vocabulary:

1. "I would like to introduce myself. I am..."	7. "I am working as a ..."
2. "Hi/Hello, I am..."	8. "I am a ..."
3. "Hi/Hello! My name is..."	9. "I studied at..."
4. "Hi/Hello! My name is...but you can call me..."	10. "I am/came here to..."
5. "I live at..."	11. "My hobbies are..."
6. "I am from..."	12. "I like..."

JOBS IN IT

a) Speaking. Which IT jobs do you know?
Examples: IT support technician, software developer, database administrator, Java Software

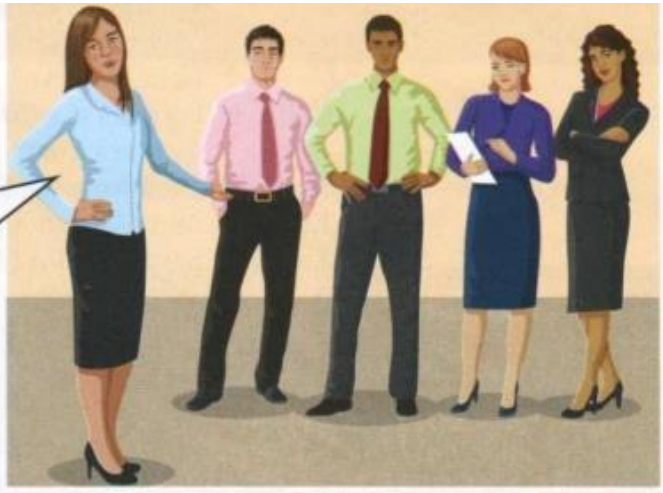
engineer,

.....

.....

b) Reading. Read this team introduction. Complete the descriptions 1-4 with the IT jobs in the box.

Hi! I'm Sylvia. I create usernames and passwords and I set firewalls.
This is Isabelle. Her job is to plan and design the network. And this is Andrew. His job is to make sure all of the computers work properly. Finally, Mark and Latika. Their area is data processing. We all work for the university. Our offices are in building 8.



database analyst IT support officer network administrator
network architect

1 Sylvia is a _____.
2 Isabelle is a _____.
3 Andrew is an _____.
4 Mark and Latika are _____.

Present simple

We use the **present simple** to talk about routines and things that are permanent or happen all the time.	*What **do** you **do**? I'**m** a programmer.*
	*What **does** she **do**? She'**s** a developer.*
	*Where **does** she **work**? She **works** for IBM in Poland. She **doesn't work** in Estonia.*
	*Where **do** they **work**? They **work** for Siemens in Egypt.*
	***Do** you **work** in IT? Yes, I **do**/No I **don't**.*

c) Listening 1. Listen to three people talking about their jobs. Complete these job descriptions.

1 Karl
Job: software _____
Responsibilities: he designs and _____ computer games.
2 Heba
Job: _____ analyst
Responsibilities: he _____ computer problems.
3 Wojtek
Job: database _____
Responsibilities: he analyses and _____ electronic data.

d) Listening 2. Listen and complete this dialogue.

Ahmed: Where (1) _____ you work, Betty?
Betty: I work for Dell in Dubai. What (2) _____ you?
Ahmed: I (3) _____ for HP in Budapest. What do you (4) _____, Milo?
Milo: I'm a (5) _____ developer. I work (6) _____ Microsoft in Prague.
Betty: Milo, do you (7) _____ Frida?
Milo: Yes, I do. What do you (8) _____ to know?
Betty: Where (9) _____ she work?
Milo: She works with (10) _____ in Prague. She designs websites for (11) _____.
Ahmed: I see. Right, let's go. The workshop starts in five minutes.

e) Listening 3. Listen to Robert, an IT worker, telling his new manager about his job. What do you think his job is? _____

Listen and tick (V) the things that usually happen in Robert´s work routine:

1 ☐ Robert checks emails.
2 ☐ Robert has emails waiting for him.
3 ☐ Robert visits people at their desks.
4 ☐ Sales people have problems.
5 ☐ Robert attends meetings.
6 ☐ Robert visits other companies.

Listen again and write these phrases in the correct place in the previous sentences.

from time to time generally hardly ever normally
occasionally usually

Expressing frequency

Adverbs of frequency (*usually*, *sometimes*, *hardly ever*, etc.) normally go before the main verb. Some adverbs (e.g. *sometimes*, *occasionally*, *normally*) can also go at the beginning or end of a sentence.	*Zafia* ***almost always*** *checks her email first thing in the morning.* *I have to call a support technician* ***occasionally****.*
Time expressions (*once a week*, *from time to time*, *all the time*, etc.) go at the beginning or end of the sentence.	*Pawel takes training courses* ***two or three times a year****.*

f) Now, what´s your dream job? Write a job description for the job of your choice.

Job:

Company to work for:

Responsibilities:

My dream job is ……

I am a ……..

I work for ……

My job is to ……

WORKING WITH COMPUTERS

Listen and complete this dialogue between Paul and Brinitha about working with computers.

Paul: Hi, Brinitha.
Brinitha: Hi, Paul.
Paul: How's it (1) _____ ?
Brinitha: Fine, fine.
Paul: What (2) _____ you (3) _____ at the moment?
Brinitha: Oh, I (4) _____ Nero.
Paul: How are you getting on?
Brinitha: Well, I (5) _____ a network. I (6) _____ Microsoft Server.
Paul: Right. Where is Jackie today? Do you know?
Brinitha: Yes. She is on a training course today. She (7) _____ about the new database system.
Paul: What about Mary and Imran? Where are they?
Brinitha: They (8) _____ in today. They have a day off.

Present continuous

We use the **present continuous** to talk about things that take place at the time of speaking and are not permanent.

I'm ***installing*** *the software.*
He's/She's ***setting up*** *a network.*
We're/They're ***working*** *at home today.*
I'm ***not setting up*** *the network.*
He's/She's ***not installing*** *the software.*
We/They ***aren't coming*** *in today.*
Are *you* ***installing*** *it now?*
What am I doing?
What are *you/they* ***doing?***
What is *he/she* ***doing?***

WEBSITES.

a) **Speaking**. What is a website? How does it work?
Which websites do you use in your work and study?
Why do you use them?

b) Reading.

Before reading, let´s discuss this question: What is the **purpose** of social networking websites like Facebook?

_____

_____

Now, read this text about different types of website. Answer the questions.

TYPES OF WEBSITE – A GUIDE FOR WEBSITE DESIGNERS

The purpose of an organisational website is to inform about an idea or event. Companies develop commercial websites to sell products or services. Entertainment websites are designed to entertain or provide fun activities. People visit news websites to obtain information. The purpose of a personal website is to provide information about an individual. Social networking websites help people to exchange personal information. Educational websites aim to share knowledge and enable online learning.

1 Why do people visit organisational websites?
2 Why do people visit company websites?
3 Why do people visit entertainment websites?
4 Why do people visit news websites?

c) <u>Vocbulary.</u>

Complete these sentences about the purpose of websites with the words in the box.

offer practise ~~present~~ promote read sell share

Example: The purpose of Nationalgeographic.com is to present information on topics.

1 People visit CNN.com to _____ international news.
2 Some websites want to _____ a service.
3 Companies use Amazon.com to _____ their products.
4 Thegreenshoppingguide.co.uk wants to _____ environmentally friendly shopping.
5 Students visit Math.com to _____ their maths.
6 English teachers join eltforum.com to _____ teaching resources.

Question words (1)

We use ***which*** to ask about things. We can use it with a noun.	***Which*** *websites do you visit/go to?* *I use Wikipedia a lot.*
We use ***what*** to ask about things.	***What*** *do you use CNN for?* *I use it to get the news.*
We use ***why*** to ask the reason for something.	***Why*** *do you use Wikipedia?* *I use Wikipedia to check information.*
We use ***when*** to ask about time.	***When*** *do you use CNN?* *In my lunchbreak.*

WEBSITE DEVELOPMENT.

1) **Speaking.** Describe something you do every day at home or at work. Use the words in the box below.

Example: 1. Sending an email.
First, click on "New email". **After that**, …. **Then**, …. **Finally**, ….

2.The process of coding.
First, …. **After that**, …. **Then,** …. **Secondly,** …. **Finally,** ….

Describing steps in a process

We use ***first***, ***next***, ***then***, ***after that*** (etc.) to describe the order of actions.	*First, do … .*	*To start, do … .*
	After that, … .	*Next, … .*
	Then, … .	
	Secondly, … .	*Thirdly, … .*
	Finally, … .	*To finish, … .*

2) **Complete this text with these words:**

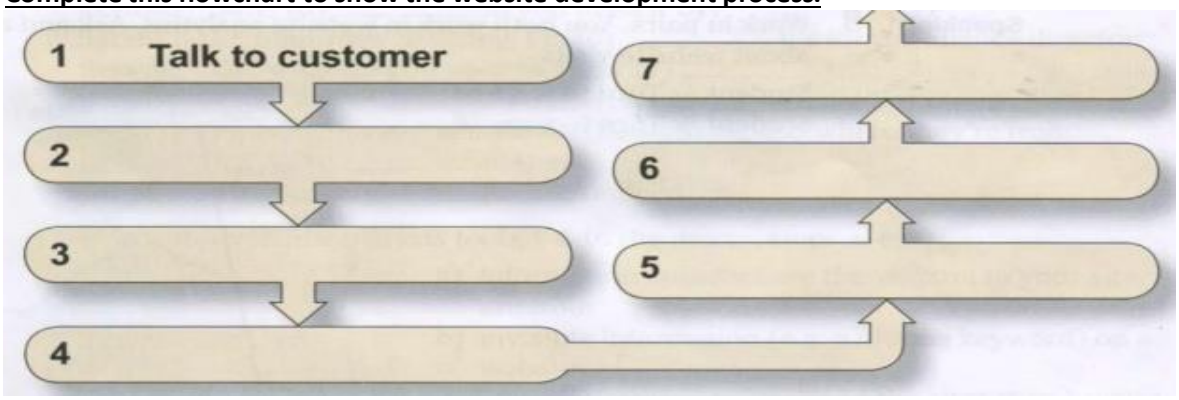
After that Finally First Next Secondly Then Thirdly

The steps in website development

(1) _____ , discuss with the customer their requirements and the target audience. Find out what features and number of pages they want on their site. (2) _____ , analyse the information from the customer. (3) _____ , create a website specification. (4) _____ design and develop the website. (5) _____ , assign a specialist to write the website content. (6) _____ give the project to programmers for HTML coding. (7) _____ , test the website.

After you publish the website, update and maintain it on an ongoing basis. Monitor customer use.

Complete this flowchart to show the website development process.



Now, describe this process to your partner using your own words.

Integration

Do the following tasks or answer the questions orally

1. Introduce yourself professionally.
2. Introduce yourself in an informal way and say something about your personal life.
3. Talk about your duties at work or your studies.
4. Make a question to a partner about their work or studies.
5. Say what your partner does at work or university.
6. What's your dream job?
7. What tasks do you normally do on your computer?
8. How often do you practice your programming skills?
9. How often do you study for university? How many subjects are you attending?
10. Invent a question with WHICH, WHERE, WHAT and WHY. Then choose one and answer it.
11. What do the following professionals do in their jobs? IT technician, website developer and database analyst.
12. Say two purposes of a website.
13. What are the positive and negative aspects of replacing people with computers at offices?
14. Do you think that it is possible that teachers are replaced by IA or computers? Why/ why not?
15. Say instructions for two of the following activities: (remember to use connectors of sequence)
    - sending an e-mail.
    - improving cybersecurity in my devices.
    - developing a website.
    - installing an app or programme.
    - developing an app.